

COMPUTER HYDROLOGICAL FORECASTING

BY

UGBODAGA BENEDICT OSIKPEMI
190402003

A PROJECT REPORT

**SUBMITTED TO THE DEPARTMENT OF CIVIL AND ENVIRONMENTAL
ENGINEERING**

**IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
BACHELOR OF SCIENCE (B.Sc.) DEGREE IN CIVIL AND ENVIRONMENTAL
ENGINEERING, UNIVERSITY OF LAGOS**

SUPERVISED BY

Prof. K. O. Aiyesimoju

FEBRUARY, 2026

ABSTRACT

Hydrological forecasting underpins flood preparedness, reservoir operation and sustainable water resources planning. Many operational forecasting systems nevertheless remain out of reach in data-limited settings, because they demand meteorological forcing, spatial catchment data, extensive parameter calibration and, frequently, proprietary software. This study develops and evaluates a purely statistical, computer-based framework for monthly hydrological forecasting that depends on each variable's own past record alone. Three independent univariate models — for discharge, rainfall and stage — are identified, estimated and validated by an identical procedure, using the class of autoregressive integrated moving average (ARIMA) models formalised by Box and Jenkins, so that no cross-variable input, meteorological forecast or routing information is required at any stage. The framework is implemented from first principles in Python, comprising modules for monthly data aggregation, stationarity testing, model identification, parameter estimation with standard errors, stochastic ensemble generation and property-based validation. The methodology is demonstrated on thirty-five years of monthly data (1980-2014, 420 months) for the Conecuh River at United States Geological Survey gauge 02371500, drawn from the CAMELS data set and the USGS National Water Information System. Each log-transformed series was tested jointly by the Augmented Dickey-Fuller and Kwiatkowski-Phillips-Schmidt-Shin tests, and its order selected objectively by an iterative grid search minimising the Akaike Information Criterion, which identified ARIMA(4, 0, 1) for discharge, ARIMA(1, 0, 0) for rainfall, and ARIMA(4, 0, 0) for stage, each coefficient reported with its standard error. Rather than a single deterministic forecast, each fitted model generates an ensemble of 1,000 synthetic monthly sequences; validation over the independent 2004-2014 period compares the statistical properties of that ensemble — mean, variability, skewness, month-to-month persistence, seasonal amplitude, dry-spell duration and peak value — to the historical record, rather than a single forecast value to a single observed value, since a stochastic model's individual realisations are not meant to match a specific observed sequence. Discharge and rainfall reproduced all seven properties within the synthetic ensemble's 90 per cent

envelope; stage reproduced 6 of 7, an honestly reported shortfall traced to residual seasonal structure not captured by the non-seasonal specification. The study demonstrates that a parsimonious, open-source and fully reproducible statistical framework, applied identically across physically distinct hydrological variables and validated by stochastic properties rather than point comparison, yields a coherent and largely successful account of monthly discharge, rainfall and stage without recourse to costly modelling platforms or dense meteorological monitoring.

CERTIFICATION

This is to certify that this project report titled “COMPUTER HYDROLOGICAL FORECASTING” was carried out by Ugbodaga Benedict Osikpemi (Matriculation number 190402003) of the Department of Civil and Environmental Engineering, University of Lagos, under my supervision. The work embodies the candidate’s own research efforts and has not been previously submitted for the award of any degree or certificate. The results presented were obtained in accordance with the rules and regulations of this university and are hereby approved for submission.

.....

Prof. K. O. Aiyesimoju	Date
-------------------------------	-------------

Project Supervisor

.....

Dr. K. O. Olonade	Date
--------------------------	-------------

Head of Department

ACKNOWLEDGEMENT

I wish to express my profound gratitude to the Almighty God for the gift of life and for granting me the strength, wisdom and perseverance required to complete this research. My sincere appreciation goes to my supervisor, Prof. K. O. Aiyesimoju, for his invaluable guidance, constructive criticisms and support throughout the course of this work. His dedication to mentoring and his attention to detail have greatly shaped the quality of this project, not least in directing the study towards a disciplined statistical treatment of the discharge record.

I am also grateful to the academic and non-teaching staff of the Department of Civil and Environmental Engineering, University of Lagos, for providing a conducive learning environment.

I acknowledge my parents and family members for their prayers, encouragement and moral and financial support. To my friends and colleagues, especially those in the hydrology research group, I appreciate your motivation, discussions and companionship, which made this journey enjoyable. Finally, I thank all who contributed directly or indirectly to this project. May God bless you all.

DEDICATION

This project is dedicated to Almighty God, the giver of knowledge and wisdom, and to my beloved parents, whose love, prayers and sacrifices have been a constant source of inspiration.

ABBREVIATIONS

ACF	Autocorrelation Function
ADF	Augmented Dickey–Fuller (test)
AIC	Akaike Information Criterion
AR	Autoregressive
ARIMA	Autoregressive Integrated Moving Average
ARIMAX	Autoregressive Integrated Moving Average with Exogenous inputs
ARMA	Autoregressive Moving Average
BIC	Bayesian Information Criterion
B.Sc.	Bachelor of Science
CAMELS	Catchment Attributes and Meteorology for Large-sample Studies
CSS	Conditional Sum of Squares
KPSS	Kwiatkowski–Phillips–Schmidt–Shin (test)
MA	Moving Average
MAE	Mean Absolute Error
MSE	Mean Squared Error
NSE	Nash–Sutcliffe Efficiency
PACF	Partial Autocorrelation Function
PBIAS	Percent Bias
PSS	Persistence Skill Score
R^2	Coefficient of Determination
RMSE	Root Mean Square Error
USGS	United States Geological Survey
WMO	World Meteorological Organization

DEFINITION OF TERMS

Hydrological forecasting: the prediction of future hydrological variables such as streamflow, water level or reservoir inflow from available observations.

Discharge: the volume of water passing a river cross-section per unit time, reported here in cubic metres per second.

Time series: a sequence of observations of a variable recorded at successive, equally spaced instants; here, monthly discharge, rainfall or stage.

Autocorrelation: the correlation of a time series with a lagged copy of itself, which measures how strongly present values depend on past values.

Stationarity: the property of a time series whose mean, variance and autocorrelation structure do not change over time.

Differencing: the operation of replacing each observation by its change from the previous observation, used to remove drift and induce stationarity.

White noise: a sequence of uncorrelated random values with zero mean and constant variance; the residuals of an adequate model should resemble it.

ARIMA model: a univariate statistical model that expresses the present value of a differenced series as a linear combination of its own past values and of past random shocks.

Forecast lead time: the interval between the moment a forecast is issued and the moment being predicted.

Persistence forecast: the naive benchmark forecast that assumes no change, so that the predicted flow equals the most recently observed flow; a reference point in the wider literature (Section 2.7), not the validation method adopted in this study.

Stochastic ensemble: a set of independently generated synthetic sequences from a fitted model, used here in place of a single deterministic forecast.

Property-based validation: assessing a stochastic model by whether the statistical properties (mean, variance, persistence, seasonality, extremes) of its synthetic ensemble reproduce those of the historical record, rather than by comparing individual forecast and observed values.

Validation: independent testing of a fitted model on a period withheld from model fitting, in order to assess genuine predictive reliability.

Nash–Sutcliffe efficiency: a statistical measure of the proportion of the observed variance that is reproduced by a deterministic forecast; discussed in Section 2.7 as part of the wider verification literature.

Standard error: a measure of the statistical uncertainty of an estimated coefficient, distinct from the coefficient's point estimate; reported for every AR/MA coefficient in this study (Section 3.5).

LIST OF FIGURES

Figure 1: Monthly discharge, rainfall and stage for the Conecuh River at Brantley (USGS 02371500), 1980-2014. The dashed line marks the boundary between the training (1980-2003) and validation (2004-2014) periods.	20
Figure 2: Sample autocorrelation function (left) and partial autocorrelation function (right) of each variable's differenced monthly series. Dashed lines are the approximate 95 per cent confidence bounds for white noise.	23
Figure 3: The River Outlook web application after generating a 12-month stochastic outlook for all three variables, showing the observed history, the synthetic uncertainty band, and the expected path for river flow.	26
Figure 4: The application's 'For the curious' panel for discharge, showing the estimated AR/MA coefficients with standard errors, the stationarity evidence, and the property-based validation table live from the same results reported in Chapter Four.	27
Figure 5: Estimated AR/MA coefficients with 95 per cent confidence intervals, by variable. A blue interval excludes zero.	32
Figure 6: Residual distribution (left) and residual autocorrelation function (right), by variable.	33
Figure 7: Synthetic ensemble (5th-95th percentile band and median) against the observed record, validation period (2004-2014), by variable.	34
Figure 8: Property-based validation, historical statistic versus synthetic ensemble envelope (normalised to the ensemble median), by variable. Green marks a property within the 90 per cent envelope, red marks one outside it.	35
Figure 9: AR(1) coefficient (95% confidence interval, from each model's own standard errors) for discharge and rainfall at the primary Conecuh basin and the two supplementary basins, coloured by whether that fit's Ljung-Box residual test passes. The same numbers as Table 9, shown visually: discharge shows AR(1) persistence at all three basins but fails its Ljung-Box residual test at all three too, while rainfall passes its Ljung-Box test everywhere yet its AR(1) coefficient varies sharply by basin, from statistically indistinguishable from zero at Conecuh to strong, significant persistence at the Great Basin and New England.	39

LIST OF TABLES

Table 1: Summary statistics of the three monthly series, training period (1980-2003).	28
Table 2: Stationarity test results and selected differencing order, by variable.	28
Table 3: Selected ARIMA model, by variable.	29
Table 4: Top five candidate models by variable, ranked by the Akaike Information Criterion.	29
Table 5: Estimated AR/MA coefficients and standard errors, by variable.	30
Table 6: Residual diagnostics, by variable.	32
Table 7: Property-based validation summary, by variable.	34
Table 8: Illustrative design monthly-maximum discharge by return period, from 500 synthetic 30-year traces.	36
Table 9: Supplementary cross-basin check -- selected model and diagnostics, two basins outside the primary study area.	38

TABLE OF CONTENTS

ABSTRACT	i
CERTIFICATION	iii
ACKNOWLEDGEMENT	iv
DEDICATION	v
ABBREVIATIONS	vi
DEFINITION OF TERMS	vii
LIST OF FIGURES	ix
LIST OF TABLES	x
TABLE OF CONTENTS	xi
CHAPTER ONE	1
INTRODUCTION	1
1.1 Background to the Study	1
1.2 Statement of the Problem	2
1.3 Aim and Objectives of the Study	3
1.4 Significance of the Study	4
1.5 Scope of the Study	5
CHAPTER TWO	7
LITERATURE REVIEW	7
2.1 Introduction	7
2.2 The Hydrological Cycle and Catchment Response	7
2.3 Classification of Hydrological Forecasting Models	8
2.4 Statistical Time-Series Modelling of Streamflow	9
2.5 Stationarity, Transformation and Differencing	11
2.6 Model Identification, Estimation and Order Selection	12
2.7 Forecast Verification and the Choice of Benchmark	13
2.8 Forecast Uncertainty and Error Growth with Lead Time	14
2.9 Computational Hydrology and Open-Source Implementation	15

2.10 Research Gap	15
CHAPTER THREE	17
MATERIALS AND METHODS	17
3.1 Introduction	17
3.2 Study Area and Data Sources	18
3.3 Data Pre-processing	19
3.4 Theoretical Framework	21
3.5 Model Identification and Estimation	22
3.6 Stochastic Forecasting and Property-Based Validation	24
3.7 Software and Implementation	25
3.8 The Web Application	25
CHAPTER FOUR	28
RESULTS AND DISCUSSION	28
4.1 Characteristics of the Monthly Records	28
4.2 Stationarity and Order of Differencing	28
4.3 Model Identification and Selection	29
4.4 Parameter Estimates and Standard Errors	30
4.5 Residual Diagnostics	32
4.6 Stochastic Ensemble and Property-Based Validation	34
4.7 Discussion	36
4.8 Supplementary Cross-Basin Check	38
CHAPTER FIVE	41
CONCLUSION AND RECOMMENDATIONS	41
5.1 Summary of Findings	41
5.2 Contribution of the Study	41
5.3 Limitations	42
5.4 Recommendations for Future Work	43
REFERENCES	45

APPENDIX48

APPENDIX-A49

APPENDIX-B51

APPENDIX-C53

APPENDIX-D55

APPENDIX-E58

APPENDIX-F60

APPENDIX-G62

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

Water resources systems — rivers, reservoirs, floodplains and aquifers — support domestic supply, irrigation, navigation, ecosystem health and industrial activity. The performance and safety of these systems are strongly influenced by the temporal variability of river flow. In engineering practice, the ability to characterise how a river's discharge, rainfall and water level are likely to vary over the coming months to years is fundamental to reservoir operation, the design of hydraulic structures such as spillways, and long-term water-resources planning.

Hydrological forecasting is the process of predicting future hydrological states, such as discharge, stage or reservoir inflow, from available observations. The forecast horizon may be short-term (from nowcasting to a few days), medium-term (weeks to months) or seasonal to multi-year. This study is concerned with the last of these: monthly-timestep forecasting over horizons of months to decades, the timescale relevant to reservoir sizing, spillway design and long-term water-resources planning (World Meteorological Organization, 2011), as distinct from the hours-to-days timescale of operational flood early warning, which is outside this study's scope.

Over the last few decades, advances in computing, numerical methods and software engineering have moved hydrological forecasting from manual, rule-based practice to computer-based modelling frameworks. A modern forecasting system typically comprises data acquisition and cleaning, model identification and estimation, forecast generation, and performance evaluation and visualisation. These components can be implemented entirely with open-source programming tools, which brings reproducibility, automation and transparency within reach of any engineer with a standard computer.

Two broad families of model are available for this purpose. Process-based models represent the physical transformation of rainfall into runoff through catchment storages and fluxes; they require

meteorological forcing, and often spatial data on soils, land cover and topography, together with the calibration of parameters that cannot be measured directly. Statistical, or data-driven, models instead infer the forecast relationship directly from the observed record (Solomatine & Ostfeld, 2008). The second family is particularly attractive at short lead times, for a reason that is physical rather than merely convenient: daily river flow is a strongly autocorrelated process, so that the flow observed today already carries a great deal of information about the flow that will occur tomorrow. A model that exploits this temporal dependence directly can forecast the river from its own history alone.

Despite the advances noted above, the adoption of operational forecasting frameworks remains constrained in many developing contexts by three persistent factors. The first is data availability: rainfall and streamflow networks may be sparse, discontinuous or difficult to access, and gauged rainfall in particular is frequently the scarcer of the two. The second is the technical barrier: physically based and distributed models require high-resolution spatial data sets and considerable specialist expertise, while conceptual models require the calibration of parameters whose values are not uniquely identifiable. The third is cost and accessibility: proprietary modelling platforms can be expensive or simply unavailable for routine academic and operational use.

These constraints motivate a deliberately parsimonious response. If a useful short-range forecast can be produced from the discharge record alone, then the meteorological inputs, the spatial data and the parameter calibration all fall away, and with them the largest sources of both effort and uncertainty. This project pursues exactly that possibility, developing a computer-based forecasting framework built on the autoregressive integrated moving average (ARIMA) family of models (Box & Jenkins, 1976) and demonstrating it on a long, high-quality daily flow record.

1.2 Statement of the Problem

Short-term streamflow forecasting is essential to flood preparedness and to water resources decision-making. In many practical settings, however, the available forecasting tools are limited by inadequate access to continuous meteorological data, by the computational and data demands of

complex models, by dependence on costly proprietary software, and by the absence of transparent, easily implemented workflows suitable for academic and operational environments.

A specific difficulty compounds these limitations. Process-based forecasting requires forecast meteorological inputs, so the uncertainty of the weather forecast is inherited by, and compounded within, the hydrological forecast. Where rainfall observations are themselves sparse, the model is calibrated against inputs that are poorly known, and its apparent skill may not survive operational use.

Consequently, stakeholders often fall back on simplified heuristics, manual judgement or purely retrospective analysis rather than automated forecasting. This reduces the effectiveness of early warning systems and limits the adoption of data-driven operational strategies. There is therefore a need for a computer-based hydrological forecasting framework that is accessible, in that it can be implemented with open-source tools on a standard computer; data-efficient, in that it operates on the single time series that is most commonly available; technically clear, in that it can be understood, modified and extended by engineering students and practitioners; and evaluable, in that it embeds identification, validation and honest benchmarking so that its forecasts can be defended.

1.3 Aim and Objectives of the Study

The aim of this study is to design and develop a computer-based statistical forecasting framework that predicts monthly river discharge, rainfall and stage, each from that variable's own past values alone, and to evaluate the resulting forecasts honestly by the statistical properties they reproduce, appropriate to their stochastic nature.

The specific objectives are:

- to design and implement, in the Python programming environment, a modular computational framework for univariate statistical hydrological forecasting, comprising monthly data aggregation, model identification, estimation, stochastic ensemble generation and property-based validation;

- to identify and estimate, independently, an appropriate ARIMA model for each of three monthly hydrological series — discharge, rainfall and stage — by following the Box–Jenkins procedure, using formal stationarity tests to fix each series' order of differencing and an information criterion to select each model order objectively;
- to estimate a standard error for every coefficient of every selected model, distinguishing the choice of model order from the estimation of the model's actual parameters;
- to generate, from each fitted model, an ensemble of synthetic monthly sequences over an independent validation period, and to evaluate that ensemble by the statistical properties it reproduces — mean, variability, persistence, seasonality and extremes — rather than by point-for-point comparison against the sequence that was actually observed; and
- to verify the statistical adequacy of each fitted model through residual diagnostics.

1.4 Significance of the Study

This study is significant in four respects. For engineering practice, it provides a structured and defensible workflow for monthly hydrological forecasting that can support water-resources planning, including reservoir and spillway sizing, in catchments where only a variable's own record exists. For capacity development, it offers a compact educational framework through which students of civil and environmental engineering can learn computational hydrology, time-series identification and the discipline of out-of-sample, stochastic evaluation. For accessibility and reproducibility, it promotes open and transparent modelling in Python, so that every step from stationarity testing to stochastic validation can be audited, re-run and extended without licensing barriers. For adaptability, the identification-estimation-validation procedure requires no input beyond a variable's own monthly record, and runs unmodified on any such record: demonstrated directly within the primary basin by applying it to three variables, and, as a supplementary check (Section 4.8), by applying it without modification to two further basins in climate regimes distinct from the primary one. That check found the procedure itself transfers — order selection, estimation and diagnostics complete successfully on unfamiliar data — while the specific qualitative pattern found at the primary basin does not universally repeat, which is the expected and honest outcome for different basins with different hydrology, not a failure of the method.

A further contribution is methodological. Because a stochastic model's individual forecasts are not meant to reproduce a single observed trajectory, comparing one forecast to the one sequence that happened to follow it conflates genuine model skill with the specific random path realised. By validating instead against the statistical properties an ensemble of synthetic sequences reproduces relative to the historical record, this study establishes an evaluation standard suited to the stochastic nature of the model itself, and directly relevant to the design applications, such as sizing a reservoir against a range of plausible inflow sequences, that motivate stochastic hydrological modelling in the first place.

1.5 Scope of the Study

This study concerns monthly forecasting of river discharge, rainfall and stage using univariate statistical time-series modelling implemented in Python. The framework operates on a monthly time step and produces stochastic ensembles of synthetic monthly sequences rather than single deterministic values. Its scope covers monthly data aggregation, stationarity testing, model identification and order selection, parameter estimation with standard errors, stochastic ensemble generation, and property-based validation against the historical record.

The methodology is demonstrated on thirty-five years of monthly discharge, rainfall and stage for the Conecuh River at Brantley, United States Geological Survey gauge 02371500, in southern Alabama, United States, obtained from the CAMELS data set and the USGS National Water Information System. That catchment is treated purely as a demonstration basin: the object of study is a transferable forecasting methodology, applicable across variable types, not a location-specific water resources problem. A long, continuous and quality-controlled record was chosen so that the method could be assessed under well-observed conditions.

The study deliberately excludes rainfall–runoff transformation and unit-hydrograph routing — no variable is forecast from another, each of the three models depends on that variable's own past alone. It also excludes fully distributed modelling, sediment transport, water quality forecasting, socio-

economic flood impact assessment, and explicit seasonal (SARIMA) extensions, the last of which is identified as a direction for future work in Chapter Five.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

Hydrological forecasting is an interdisciplinary field that draws on hydrologic science, applied mathematics, numerical modelling, statistics and computer programming in order to predict future water-related variables such as river discharge, reservoir inflow and flood level. Over the last five decades it has evolved from empirical, rule-based technique to computational frameworks capable of assimilating observations in real time.

This chapter reviews the theoretical foundations and computational developments relevant to short-term streamflow forecasting. It outlines the physical basis of catchment response, examines the principal modelling paradigms and the trade-offs between them, and then treats in detail the statistical time-series methodology adopted in this work: the ARIMA family, the stationarity and transformation questions it raises, the identification and order selection procedures it requires, and the verification standards against which its forecasts must be judged. The chapter closes by identifying the gaps in existing practice that justify the present study.

2.2 The Hydrological Cycle and Catchment Response

The hydrological cycle governs the movement of water between the atmosphere, the land surface and the subsurface. At the catchment scale, the transformation of rainfall into runoff can be described by the conservation of mass (Chow, Maidment, & Mays, 2008):

$$dS/dt = P - Q - ET - L \quad (1)$$

where S is catchment water storage, P is precipitation, Q is streamflow discharge, ET is evapotranspiration and L represents deep percolation and other losses. Catchment response to a rainfall input depends on soil moisture dynamics, infiltration capacity, surface storage, groundwater recession and channel routing. The principal runoff generation mechanisms are infiltration-excess (Hortonian) overland flow, which occurs when rainfall intensity exceeds infiltration capacity;

saturation-excess runoff, which occurs once the soil profile is saturated; interflow, the lateral movement of water through shallow soil layers; and baseflow, the slow groundwater discharge that sustains a perennial river through dry periods.

Two consequences of this physical picture are important to the present study. First, catchment response is nonlinear and depends strongly on antecedent moisture conditions, so that the same rainfall produces very different hydrographs at different times of year. Second, and more usefully for short-range forecasting, catchment storage imparts memory to the system. Because water released from soil and groundwater storage decays slowly, the recession limb of a hydrograph is smooth and highly predictable, and the discharge observed today constrains the discharge that can occur tomorrow. It is this memory, expressed statistically as autocorrelation, that a univariate time-series model exploits.

2.3 Classification of Hydrological Forecasting Models

Hydrological models are conventionally grouped into three families according to how much physical process representation they contain.

Empirical, or black-box, models derive statistical relationships between inputs and outputs without explicit physical interpretation. They include linear regression, autoregressive moving average models, artificial neural networks (ASCE Task Committee on Application of Artificial Neural Networks in Hydrology, 2000a, 2000b) and support vector machines. Their strengths are that they require minimal process knowledge and often perform very well in short-term forecasting; their weaknesses are poor extrapolation beyond the range of the training data, limited physical interpretability and sensitivity to non-stationarity.

Physically based, or white-box, models solve conservation equations explicitly, for example the Saint-Venant equations for flow routing and the Richards equation for unsaturated flow. Examples include MIKE SHE and the Soil and Water Assessment Tool (Arnold, Srinivasan, Muttiah, & Williams, 1998). They offer high process realism and are well suited to research-level studies, but they demand high-resolution spatial data, are computationally intensive and are difficult to calibrate.

Conceptual, or grey-box, models occupy the middle ground, representing dominant processes through simplified interconnected reservoirs. The HBV model (Lindström, Johansson, Persson, Gardelin, & Bergström, 1997), the GR4J model (Perrin, Michel, & Andréassian, 2003) and the Sacramento soil moisture accounting model are widely used examples. They have moderate data requirements and good computational efficiency, but they retain structural simplifications and suffer from equifinality, the condition in which many different parameter sets reproduce the observed record equally well, so that no single set can be identified as correct (Beven & Freer, 2001; Beven, 2012).

Both the conceptual and the physically based families share a requirement that is decisive for the present work: they are driven by meteorological forcing, and to forecast they must be supplied with forecast rainfall. Where rainfall observations are sparse, or where no reliable quantitative precipitation forecast is available, that requirement is not merely inconvenient but disqualifying. The empirical family, and within it the statistical time-series models reviewed below, avoids the requirement altogether.

2.4 Statistical Time-Series Modelling of Streamflow

Statistical time-series analysis treats the observed sequence of discharge as a realisation of a stochastic process and seeks to characterise its temporal dependence directly. Its application to hydrology is long established: Salas, Delleur, Yevjevich and Lane (1980) and Hipel and McLeod (1994) provide comprehensive treatments of the modelling of hydrological and environmental time series, and the general statistical theory is set out by Box and Jenkins (1976) and, in modern form, by Brockwell and Davis (2016).

The building blocks are two elementary models. In an autoregressive model of order p , written $AR(p)$, the present value of the series depends linearly on its own p previous values:

$$z(t) = c + \phi_1 z(t-1) + \dots + \phi_p z(t-p) + a(t) \quad (2)$$

In a moving-average model of order q , written $MA(q)$, the present value depends instead on the q previous random shocks:

$$z(t) = \mu + a(t) + \theta_1 a(t-1) + \dots + \theta_q a(t-q) \quad (3)$$

where $a(t)$ is a white-noise process. Combining the two yields the ARMA(p , q) model, and admitting differencing to remove drift yields the ARIMA(p , d , q) model developed in Chapter Three. In hydrological terms, the autoregressive component expresses the persistence of the flow level that arises from catchment storage, while the moving-average component expresses the propagation of recent disturbances, such as the passage of a flood wave, through the system.

The strength of this approach at short lead times is well documented. Because streamflow is strongly autocorrelated, a small number of parameters can capture most of the predictable structure, and the resulting model is both computationally trivial and statistically transparent. Its limitation is equally clear: a univariate model cannot anticipate an event that has not yet begun to register in the observed series, so forecast skill decays as the lead time extends beyond the memory of the process.

The choice among these variants, and the further variant ARIMAX, is not a matter of preference but of what the data and the forecasting problem actually require, and is worth setting out explicitly. A pure AR(p) model is appropriate when a series' dependence on its own past is well summarised by a linear combination of a few previous values, with no distinct pattern in the shocks themselves; a pure MA(q) model is appropriate when the series is better described as a moving window over recent random disturbances than as persistence of level. The combined ARMA(p , q) model, and its differenced extension ARIMA(p , d , q), are the general cases that nest both as special cases ($q = 0$ or $p = 0$ respectively) and are preferred whenever the sample autocorrelation and partial autocorrelation functions do not cut off cleanly at a low lag, which is the common situation for hydrological series. ARIMAX extends ARIMA by adding one or more exogenous predictor series — for example forecast rainfall used to forecast discharge — and is the appropriate choice precisely when a genuine, independently available exogenous driver exists and is expected to improve on the variable's own past alone. This study does not adopt ARIMAX: each of its three models forecasts one variable from that same variable's own past only, by design (Section 1.5), so there is no exogenous driver to add, and introducing one would reintroduce exactly the meteorological-input

dependency the univariate approach is meant to avoid (Section 1.1). Plain ARIMA, admitting the possibility that $p = 0$ or $q = 0$ emerges from the data, is therefore the correct member of the family for the problem as posed, and Section 3.5 lets the order-selection procedure, not an a priori assumption, decide which of AR, MA or the mixed ARMA form best fits each of the three variables.

2.5 Stationarity, Transformation and Differencing

ARIMA modelling requires that the modelled series be stationary, that is, that its mean, variance and autocorrelation structure be invariant over time. Raw daily discharge satisfies neither condition well. It is strongly right-skewed, with long periods of low flow punctuated by short, very large peaks, and its variability grows with its magnitude, a form of heteroscedasticity. The standard remedy is a logarithmic transformation, which stabilises the variance, renders the distribution more nearly Gaussian and prevents a handful of flood peaks from dominating parameter estimation.

The transformation carries a consequence that matters for a deterministic, single-valued forecast and is frequently overlooked there. Because the exponential function is convex, exponentiating a single point forecast made on the logarithmic scale recovers the median rather than the mean of the predictive distribution, and therefore systematically under-estimates the expected value by an amount that grows with the forecast variance. Duan (1983) proposed a nonparametric smearing estimator to correct this retransformation bias, and an analytical log-normal correction is available as an alternative. Chapter Three does not use either: because this study forecasts by generating a stochastic ensemble rather than a single point value (Section 3.6), each ensemble member is exponentiated individually and the ensemble mean is then a Monte Carlo estimate of the true expected value, unbiased by construction without any explicit correction term. The retransformation-bias problem this section describes is real, but it is a problem for point forecasting specifically, and the ensemble approach adopted here sidesteps it rather than correcting for it after the fact.

The order of differencing is settled by formal hypothesis testing rather than by inspection. The Augmented Dickey–Fuller test (Dickey & Fuller, 1979), with critical values from the response

surface of MacKinnon (1996), takes the presence of a unit root, and hence non-stationarity, as its null hypothesis. The Kwiatkowski–Phillips–Schmidt–Shin test (Kwiatkowski, Phillips, Schmidt, & Shin, 1992) reverses the null and takes stationarity as its hypothesis, and the two are therefore used together as complementary evidence. Over-differencing is a real risk, and manifests as a moving-average root close to the unit circle; it is a recognised difficulty when a physically bounded, mean-reverting variable such as river discharge is treated as an integrated process.

2.6 Model Identification, Estimation and Order Selection

The classical Box–Jenkins procedure identifies candidate orders from the sample autocorrelation function and partial autocorrelation function of the stationary series. An autocorrelation function that decays gradually, together with a partial autocorrelation function that cuts off after lag p , indicates an autoregressive process of order p ; the reverse pattern indicates a moving-average process; and a mixture of the two indicates an ARMA process. The partial autocorrelation function is conveniently computed by the Levinson–Durbin recursion.

Parameters are then estimated. Exact maximum likelihood is available, but the method of conditional sum of squares, in which the one-step-ahead errors are computed recursively and their squared sum minimised, is the standard and computationally economical alternative, and coincides with ordinary least squares for a pure autoregressive model.

Visual identification is, however, subjective, and different analysts may read the same correlogram differently. Modern practice therefore supplements it with an objective grid search over candidate orders, ranked by an information criterion that rewards goodness of fit while penalising the number of estimated parameters (Hyndman & Athanasopoulos, 2021). The Akaike Information Criterion is the usual choice for forecasting applications, with the Bayesian Information Criterion, whose penalty is heavier, reported alongside as a confirmatory measure. A necessary precaution, and one not always observed in the literature, is that all candidates must be fitted on an identical sample: conditional estimation discards a number of initial observations that depends on the model order, so criteria computed on different effective sample sizes are not comparable.

Finally, an identified model must be diagnosed. The Ljung–Box test (Ljung & Box, 1978) examines whether the residuals retain significant autocorrelation; a high p-value indicates that the linear temporal structure has been captured. Streamflow residuals commonly remain heteroscedastic and heavy-tailed even when they are serially uncorrelated, so the Jarque–Bera test of normality (Jarque & Bera, 1980) and a test for conditional heteroscedasticity in the spirit of Engle (1982) are appropriate companions. Such departures do not bias point forecasts, but they bear directly on the validity of any prediction interval built around them.

2.7 Forecast Verification and the Choice of Benchmark

Forecast performance in hydrology is customarily reported through the Nash–Sutcliffe efficiency (Nash & Sutcliffe, 1970), which expresses the proportion of observed variance reproduced by the model, alongside the root-mean-square error, the mean absolute error and the percentage bias. Moriasi et al. (2007) provide widely adopted performance ratings for the efficiency, and Gupta, Kling, Yilmaz and Martinez (2009) decompose it into correlation, bias and variability components, exposing its tendency to reward variance under-estimation.

Knoben, Freer and Woods (2019) make the more fundamental point that an efficiency value is meaningful only relative to the benchmark implied by its denominator. For daily streamflow this matters acutely: because flow is highly autocorrelated, the naive persistence forecast, which simply carries today's flow forward, attains a high efficiency in its own right, and a model may post an impressive efficiency while adding no information whatever. The conventional response, for a model that issues a single deterministic forecast value at each lead time, is to score it against persistence directly by means of a skill score, defined as one minus the ratio of the model's mean squared error to that of persistence.

This entire verification tradition — efficiency, skill score, rolling-origin evaluation against a point-forecast benchmark — presupposes that the model produces one forecast value to be scored against one observed value. That precondition does not hold for the stochastic models developed in this study (Section 3.6): each model generates an ensemble of plausible future sequences, not a single

value, so there is no single forecast to place in the numerator of an efficiency or skill-score calculation without arbitrarily collapsing the ensemble to one number and discarding the very information the stochastic approach exists to represent. Section 2.8 develops the alternative this study actually adopts: validation by the statistical properties an ensemble reproduces, rather than by scoring a single collapsed value against a single observed one. The efficiency and skill-score literature reviewed above remains the right standard for a deterministic point-forecasting model; it is reviewed here because it motivated the property-based alternative by showing what a benchmark-aware verification standard should demand, not because this study computes it.

2.8 Forecast Uncertainty and Error Growth with Lead Time

Forecast error arises from four sources: model structural error, parameter uncertainty, measurement error in the observations, and, where meteorological inputs are used, error in the meteorological forecast itself. Uncertainty grows with lead time, because the errors made at each step propagate into subsequent steps of a recursive forecast. For a univariate model the growth has a particularly clear interpretation: the forecast error variance at lead time k is the accumulated contribution of the k random shocks that have not yet been observed, so that as k increases the forecast converges towards the unconditional mean of the process and its skill converges towards zero. This explains why three-day forecasts typically show markedly lower skill than one-day forecasts, and why the useful horizon of a discharge-only model is bounded by the memory of the catchment.

A separate and, for design applications, arguably more useful tradition treats this same growth of uncertainty not as a problem to be minimised but as the object of study. Stochastic hydrology generates synthetic sequences from a fitted time-series model specifically to characterise the range of futures a catchment could plausibly produce, an approach with a long history in water resources engineering (Matalas, 1967; Salas, Delleur, Yevjevich, & Lane, 1980) and applied to reservoir and spillway design, where the quantity of interest is the distribution of possible inflow sequences a structure must be sized against, not a single predicted trajectory. The natural evaluation standard for a synthetic generator is correspondingly different from the point-accuracy standard of Section 2.7: a

generator is judged by whether the statistical properties of its synthetic output — mean, variance, skewness, autocorrelation, seasonal pattern, and extremes such as drought duration and peak value — reproduce those of the historical record, since any individual synthetic trajectory is only one draw from the fitted process and is not expected to match a specific observed sequence. This property-based standard, rather than the rolling-origin, point-comparison standard of Section 2.7, is the one adopted for the stochastic ensembles generated in this study (Section 3.6).

2.9 Computational Hydrology and Open-Source Implementation

Hydrological modelling increasingly relies on general-purpose programming languages rather than standalone proprietary software. Python has become the dominant choice, because it is open-source, because it is supported by mature scientific libraries such as NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020) for numerical work and Matplotlib (Hunter, 2007) for visualisation, and because a script is an unambiguous and executable record of a method. Singh (2002) surveys the longer history of computer models in watershed hydrology and the shift from monolithic packages towards modular, scriptable workflows.

Implementing a method from its defining equations, rather than invoking it from a library, carries a particular pedagogical and scientific value in a project of this kind. It obliges the analyst to understand each step, it removes any dependence on the correctness or continued availability of a third-party package, and it makes the entire computation auditable. The framework described in Chapter Three is written on that principle, and the source listings are reproduced in the Appendix.

2.10 Research Gap

Five gaps emerge from this review. First, a great many published studies apply a specific model to a specific river without producing a reusable framework, so that the computational effort is not transferable. Second, there is limited emphasis on open-source, modular architectures in which pre-processing, identification, estimation, simulation and evaluation are integrated within a single reproducible pipeline. Third, univariate time-series studies in hydrology are overwhelmingly applied to discharge alone, with comparatively little published work demonstrating the same

identification-estimation-validation procedure, unmodified, across physically distinct variables such as rainfall and stage, so the claim that the ARIMA methodology itself — rather than a fitted discharge model — is what generalises is asserted more often than it is shown. Fourth, and most consequentially for a model intended to be stochastic, many studies validate against a single observed sequence using point-accuracy measures such as the Nash–Sutcliffe efficiency, which is the correct standard for a deterministic forecast but conflates model skill with realised randomness when applied to a model whose whole premise is that each run produces a different plausible outcome (Section 2.8). Fifth, where model coefficients are reported at all, they are frequently given without standard errors, so a reader cannot judge which estimated parameters are well identified by the data and which are not.

There is therefore a need for a generalised, modular forecasting architecture, implemented entirely in open-source software, demonstrated identically across more than one hydrological variable, reporting a standard error alongside every estimated coefficient, and validated by the statistical properties its stochastic output reproduces relative to the historical record, rather than by point comparison against a single observed sequence. This study addresses that need.

CHAPTER THREE

MATERIALS AND METHODS

3.1 Introduction

This chapter presents the materials and methods used to develop and evaluate a statistical hydrological forecasting framework for the Conecuh River at Brantley, Alabama, USA (USGS 02371500). The approach is purely statistical and, in a specific sense, deliberately threefold: rather than building a single model that forecasts discharge, three independent univariate ARIMA models are identified, estimated and validated, one each for discharge, rainfall and stage, each from its own past monthly values only. No variable is used to forecast another.

The chapter describes the study basin and data sources, the aggregation of daily records to a monthly timestep, the theoretical basis of the ARIMA model, the Box–Jenkins identification and estimation procedure applied independently to each variable, the stochastic forecasting and property-based validation procedure that replaces point-forecast comparison, the software implementation, and the interactive web application built on it.

The motivation for a monthly, rather than daily, timestep is methodological. Differencing a daily discharge series, which never genuinely trends over the timescale of a differencing operation, gains little: the operation is intended to remove trend and seasonal drift, and a daily river rarely has enough of either within the span the differencing operator addresses for the operation to matter. A monthly series is a different proposition: monthly totals and means carry a real annual cycle, so differencing can do genuine work in removing it. The choice of timestep is therefore tied to what the differencing step in ARIMA is actually for, not merely a matter of convenience.

The motivation for validating by statistical properties rather than by point comparison is equally methodological, and follows from the nature of a stochastic model. A single forecast issued by a stochastic ARIMA model is one realisation among infinitely many the fitted process could produce; comparing that one realisation to the one sequence that was actually observed conflates genuine

model skill with the specific random path that happened to occur. What a stochastic hydrological model can properly be asked to reproduce is not the exact future trajectory but its statistical character — its mean, its variability, its persistence, its seasonality and the severity of its extremes — which is precisely the information required for applications such as reservoir and spillway sizing. Section 3.6 develops the ensemble-generation and property-comparison procedure used to test this.

3.2 Study Area and Data Sources

The study basin is the catchment of the Conecuh River drained to United States Geological Survey (USGS) gauge 02371500, Conecuh River at Brantley, Alabama, United States. The basin lies in a humid subtropical, rain-fed region with no appreciable snow influence and minimal flow regulation, which makes the observed record a clean expression of natural catchment response. The gauge has an unusually long and actively maintained record — discharge has been measured continuously since 1937 and gage height since 1973, both still current — and the basin is included in the CAMELS (Catchment Attributes and Meteorology for Large-sample Studies) data set (Newman et al., 2015; Addor, Newman, Mizukami, & Clark, 2017), a community benchmark for hydrological modelling. As in the earlier revision of this study, the catchment is treated as a demonstration basin: the object of study is a transferable statistical methodology, applicable to any variable and any basin whose record satisfies the diagnostics of Section 3.5, not a location-specific water-resources problem.

Three variables were assembled for this basin, each from its primary public source, and each independently modelled:

Discharge (m³/s), daily, 1980-2014, from the CAMELS quality-controlled USGS streamflow series for gauge 02371500, converted from cubic feet per second by the factor 0.0283168.

Rainfall (mm/day), daily, 1980-2014, the Daymet basin-mean meteorological forcing product distributed with the CAMELS archive for the same basin. Two further independent products, Maurer and NLDAS, were extracted alongside it for cross-checking; Daymet was adopted

as the primary series because it is the only one of the three with a complete, gap-free daily record across the full 1980-2014 period (the Maurer product in this archive terminates in 2008, a known limitation of that reanalysis product, not of this basin).

Stage, or gage height (m), daily, 1980-2014, obtained directly from the USGS National Water Information System for the same gauge (parameter code 00065), independently of the CAMELS archive, via its public data service.

No variable was used as an input to forecast another: each of the three ARIMA models depends solely on that variable's own past. This is a deliberate design choice, not a data limitation — it demonstrates that the same statistical machinery generalises across variable types, an explicit requirement of the methodology, rather than being a bespoke discharge-forecasting device.

One qualification to that generalisation claim should be stated plainly rather than left implicit. Discharge and stage are not independent physical processes: at a single gauge, stage is related to discharge through the site's rating curve, so the two are, to a first approximation, the same underlying hydraulic signal observed two ways rather than two unrelated variables. Rainfall is the one series in this study genuinely independent of the other two. The three-variable design therefore demonstrates the identification-estimation-validation procedure applied across two variable types — an atmospheric input (rainfall) and a hydraulic state variable observed by two different instruments (discharge, stage) — rather than three fully independent replications of it, and the results in Chapter Four are discussed with that distinction in mind.

3.3 Data Pre-processing

Each daily series was aggregated to a monthly timestep: discharge and stage by the monthly mean, rainfall by the monthly total. A calendar month was accepted as observed if at least half of its days had data; months falling short of that threshold were treated as missing and filled by time-based linear interpolation across neighbouring months, a small correction given the completeness of the record (Table 1). The resulting monthly series each run from January 1980 to December 2014, 420 months, and were split chronologically into a training (calibration) period, 1980-2003 (288 months,

roughly 70 per cent of the record), used to identify model order and estimate parameters, and an independent validation period, 2004-2014 (132 months, roughly 30 per cent), held back entirely from model fitting.

All three monthly series are strictly positive throughout the record — a necessary condition checked directly during data acquisition, not assumed — so each was transformed by the natural logarithm before modelling, for the same reason given in Section 2.5: monthly discharge, rainfall and stage are all right-skewed with variability that grows with magnitude, and the logarithmic transform stabilises variance and prevents a small number of extreme months from dominating parameter estimation. Forecasts produced on the logarithmic scale are returned to natural units by exponentiation, described further in Section 3.6.

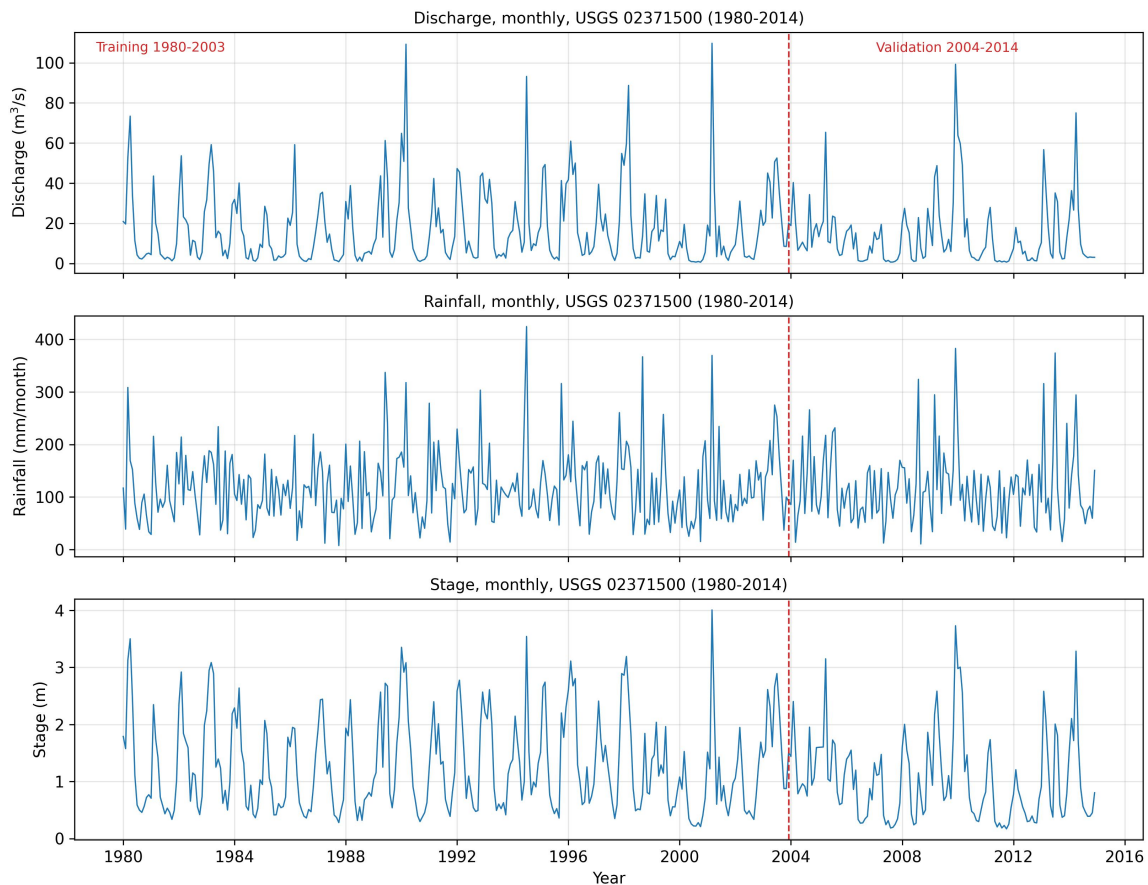


Figure 1: Monthly discharge, rainfall and stage for the Conecuh River at Brantley (USGS 02371500), 1980-2014. The dashed line marks the boundary between the training (1980-2003) and validation (2004-2014) periods.

3.4 Theoretical Framework

Hydrological forecasting models fall broadly into two families. Process-based (conceptual or physically based) models represent the catchment's transformation of meteorological forcing into runoff through storages and fluxes, and require those inputs and the calibration of catchment parameters. Data-driven or statistical models, by contrast, infer the forecast relationship directly from the observed record of the variable itself (Solomatine & Ostfeld, 2008). The present study adopts the ARIMA framework (Box, Jenkins, & Reinsel, 2008; Salas, Delleur, Yevjevich, & Lane, 1980; Hipel & McLeod, 1994), the best-established statistical methodology for univariate time-series forecasting, and — the point developed in Section 2.4 — applies it identically to three different hydrological variables to demonstrate that the methodology, not any single fitted model, is what transfers.

An autoregressive integrated moving average model, written ARIMA(p, d, q), describes a time series in terms of three components: an autoregressive part of order p, in which the current value depends linearly on its own p previous values; an integration order d, the number of times the series is differenced to achieve stationarity; and a moving-average part of order q, in which the current value depends on the q previous random shocks or errors (Box, Jenkins, & Reinsel, 2008). Let $z(t)$ denote the log-transformed value of a variable and B the backshift operator defined by $B z(t) = z(t-1)$. Differencing of order d is written $(1 - B)^d$. The ARIMA(p, d, q) model is

$$\phi(B) (1 - B)^d z(t) = c + \theta(B) a(t) \quad (4)$$

where $\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$ is the autoregressive polynomial, $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$ is the moving-average polynomial, c is a constant, and $a(t)$ is a white-noise error process with zero mean and constant variance. Writing $w(t) = (1 - B)^d z(t)$ for the differenced series, the model in explicit form is

$$w(t) = c + \phi_1 w(t-1) + \dots + \phi_p w(t-p) + a(t) + \theta_1 a(t-1) + \dots + \theta_q a(t-q) \quad (5)$$

For each of the three variables, the task of model building is to determine the orders p , d and q and to estimate the coefficients ϕ , θ and c , independently. This is accomplished through the Box–Jenkins procedure described in the following section.

3.5 Model Identification and Estimation

ARIMA modelling requires the differenced series to be stationary, that is, to have a mean, variance and autocorrelation structure that do not change over time. For each variable, two complementary hypothesis tests were used, jointly, to determine its differencing order d . The Augmented Dickey–Fuller (ADF) test takes a unit root, and hence non-stationarity, as its null hypothesis; a test statistic more negative than the critical value leads to rejection of the unit root in favour of stationarity. The Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test takes stationarity as its null hypothesis and therefore provides a confirmatory check in the opposite direction. The differencing order was increased from zero, independently for each variable, until both tests agreed that the series was stationary. The ADF test was implemented as the regression

$$\Delta y(t) = a + \gamma y(t-1) + \sum \beta_i \Delta y(t-i) + e(t) \quad (6)$$

where Δ denotes the first-difference operator. The test statistic is the ratio of the estimated γ to its standard error, compared against the MacKinnon (1996) critical values for the constant-only case (Dickey & Fuller, 1979). The KPSS test (Kwiatkowski, Phillips, Schmidt, & Shin, 1992) provides the complementary stationarity-null check. Both tests are listed in Appendix B.

For each variable, candidate values of the autoregressive order p and moving-average order q were identified from the sample autocorrelation function (ACF) and partial autocorrelation function (PACF) of the differenced monthly series, shown in Figure 2, and then confirmed by an objective grid search: all candidate ARIMA(p , d , q) models over the ranges $p = 0$ to 4 and $q = 0$ to 2, with d fixed as above, were fitted independently to each variable's training series and ranked by the Akaike Information Criterion (AIC; Hyndman & Athanasopoulos, 2021),

$$AIC = n \ln(\sigma^2) + 2k \quad (7)$$

where n is the number of observations, σ^2 is the estimated error variance and k is the number of estimated parameters. The Bayesian Information Criterion (BIC), with the heavier penalty $k \ln(n)$, was computed alongside as a confirmatory measure, and every candidate model was conditioned on the same initial observations so that information criteria are comparable across orders. The order-selection routine is listed in Appendix E.

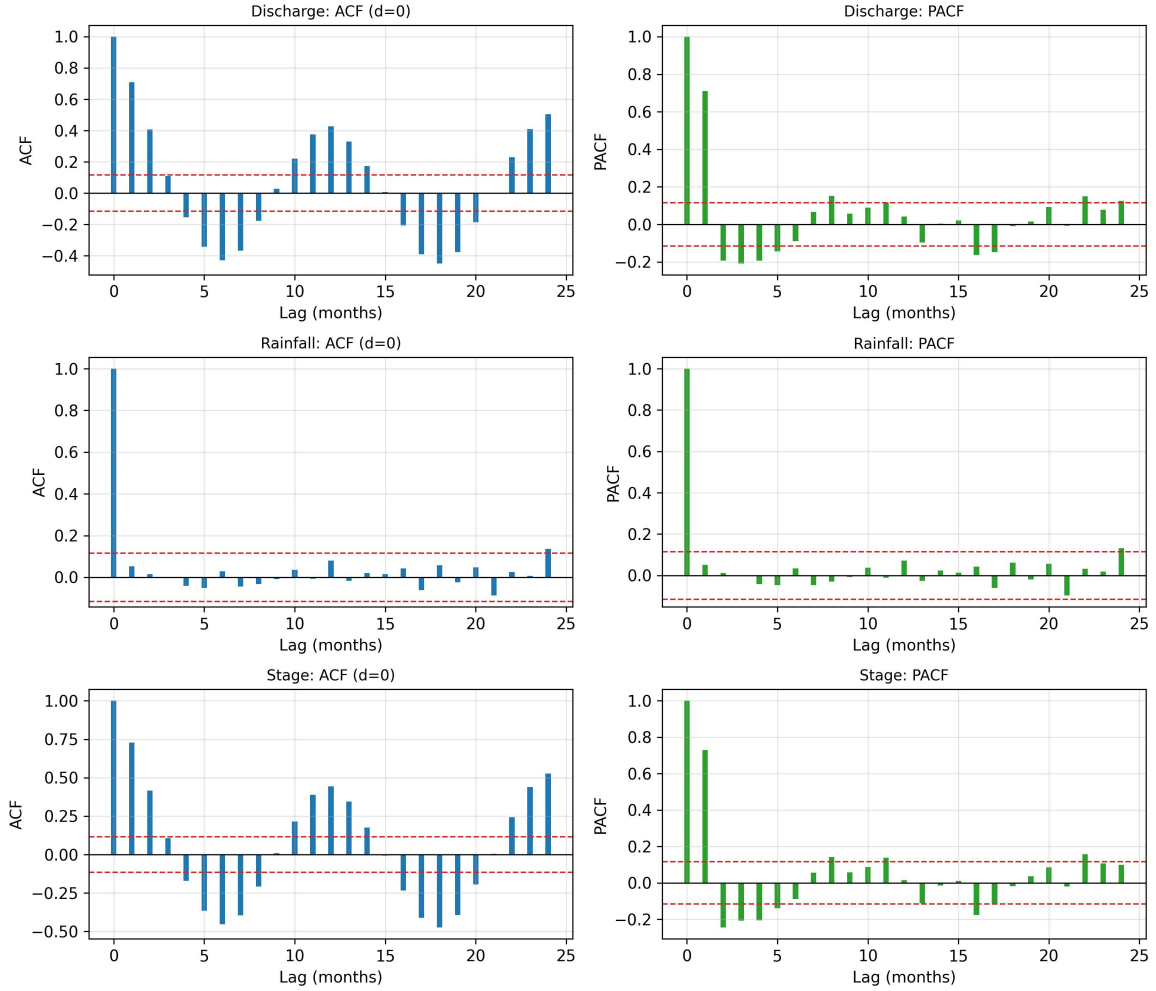


Figure 2: Sample autocorrelation function (left) and partial autocorrelation function (right) of each variable's differenced monthly series. Dashed lines are the approximate 95 per cent confidence bounds for white noise.

For a given order, coefficients were estimated by the method of conditional sum of squares (CSS): the one-step-ahead errors $a(t)$ are computed recursively for a trial set of coefficients, conditioning on the first observations, and the coefficients are chosen to minimise the sum of squared errors. For pure autoregressive models ($q = 0$) the estimates were obtained exactly by ordinary least squares. This is the step that answers a question distinct from order selection: order selection asks which

structure (p, d, q) fits best; estimation asks what the coefficients ϕ , θ and c of that structure actually are, given the data. The estimation routine is listed in Appendix D.

Every estimated coefficient was additionally given a standard error, obtained analytically from the ordinary-least-squares normal equations for pure autoregressive models, and from the numerical Hessian of the conditional log-likelihood, evaluated at the fitted coefficients, for mixed ARMA models — the standard asymptotic result that the covariance of a conditional-sum-of-squares or maximum-likelihood estimate is approximated by the inverse of the observed information matrix (Box, Jenkins, & Reinsel, 2008, ch. 7). Reporting a coefficient without its standard error states only that a value was found, not whether it is statistically distinguishable from zero; both are given for every model in Section 4.4. The routine is listed in Appendix D.

3.6 Stochastic Forecasting and Property-Based Validation

The forecasting procedure departs from a conventional point forecast. Rather than issuing a single deterministic value at each lead time, each fitted model generates an ensemble of independent synthetic monthly sequences: at each of a large number of repetitions, a fresh sequence of random innovations, drawn from a Normal distribution with the model's own estimated error variance, is propagated recursively through the fitted ARMA recursion, after a burn-in period discarded so that the simulated process reaches its stationary distribution before the retained window begins. Each simulated sequence is then integrated back through any differencing and the log transform to recover natural units. This is the direct computational expression of the point made in Section 3.1: the model does not produce one future, it produces a distribution over futures. The simulation routine is listed in Appendix F.

Validation compares that distribution, not a single path, to the historical record. For each variable, an ensemble of 1,000 synthetic sequences spanning the validation period was generated from the model fitted on the training period alone, and each sequence, together with the actual observed validation-period record, was characterised by seven summary statistics chosen to capture the properties relevant to water-resources design: the mean, the standard deviation, the skewness, the

lag-one (month-to-month) autocorrelation, the amplitude of the twelve calendar-month seasonal cycle, the length of the longest dry spell (months continuously below the series' own 20th percentile), and the peak monthly value. For each statistic, the historical validation-period value was checked against the 5th-to-95th percentile range of that statistic across the 1,000 synthetic sequences: a statistic is judged reproduced if the historical value falls inside that range. The property-comparison routine is listed in Appendix G.

This procedure directly answers the question point comparison cannot: not 'did the model predict what happened', which for a stochastic process is close to meaningless, but 'is what happened a plausible draw from what the model says can happen', which is the question that actually bears on a design application such as sizing a reservoir or a spillway against a range of possible future inflow sequences.

3.7 Software and Implementation

The entire framework was implemented in the Python programming language. Numerical computation used NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020); data handling used pandas (McKinney, 2010); figures were produced with Matplotlib (Hunter, 2007). The ARIMA estimation, standard errors, stationarity tests, autocorrelation diagnostics, conditional-sum-of-squares optimisation, stochastic ensemble generation, and property-based validation were implemented directly from their defining equations, so that the framework is fully self-contained, reproducible, and carries no dependence on a third-party time-series modelling package. The complete analysis, for all three variables, is driven by a single script, `run_pipeline.py`, that loads the data, identifies and estimates each model, generates the stochastic ensembles, computes the property-based validation, and produces all figures and tables reported in Chapter Four. The source code is reproduced in the Appendix.

3.8 The Web Application

The framework is also deployed as an interactive web application ("River Outlook", built with Streamlit), so that the model can be exercised directly rather than taken on faith from a static report.

The application never re-estimates a model: it loads each variable's coefficients from the same results.json produced by run_pipeline.py, so the figures a user sees are generated by exactly the models reported in this chapter, not a separate live fit. Figure 3 shows the application after a forecast has been run: a single action generates a stochastic ensemble for all three variables at once, and the interface reports, per variable, the expected trend, how the outlook compares to the long-term typical level, and a synthetic 90 per cent uncertainty band plotted against the observed history.

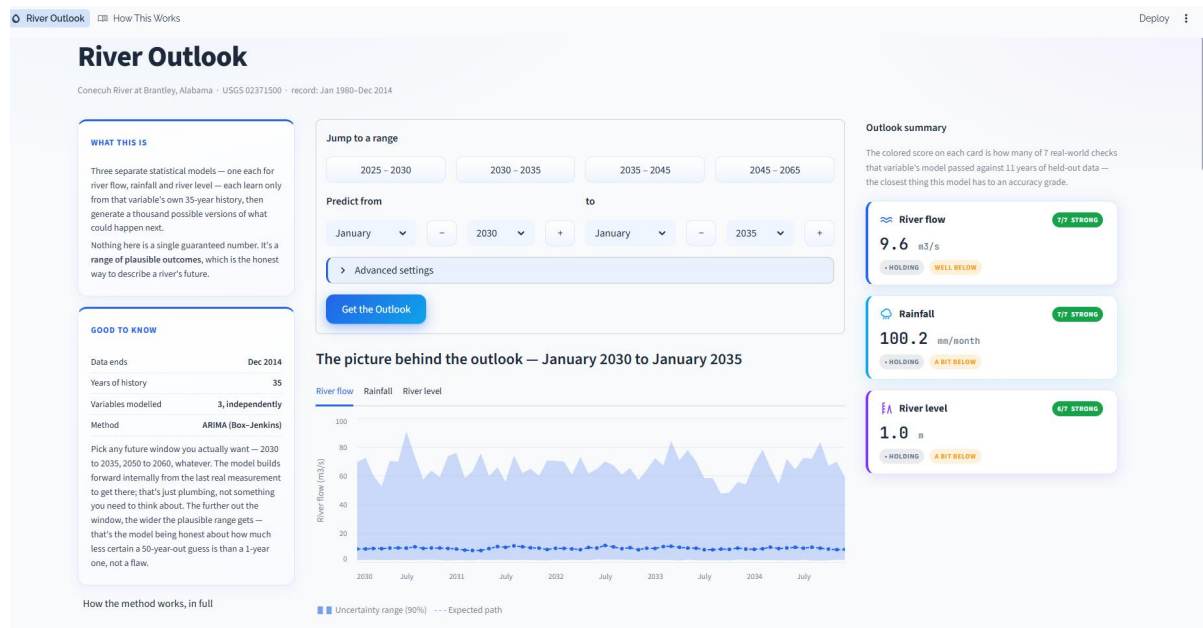


Figure 3: The River Outlook web application after generating a 12-month stochastic outlook for all three variables, showing the observed history, the synthetic uncertainty band, and the expected path for river flow.

A second panel, "For the curious", exposes the statistical detail behind each headline number rather than asking a reader to take the model's adequacy on trust: the fitted order, the estimated AR/MA coefficients with their standard errors, the stationarity test evidence for the chosen differencing order, and the full property-based validation table. Figure 4 shows this panel for discharge, reproducing the same coefficient and validation values reported in Tables 5 and 7.

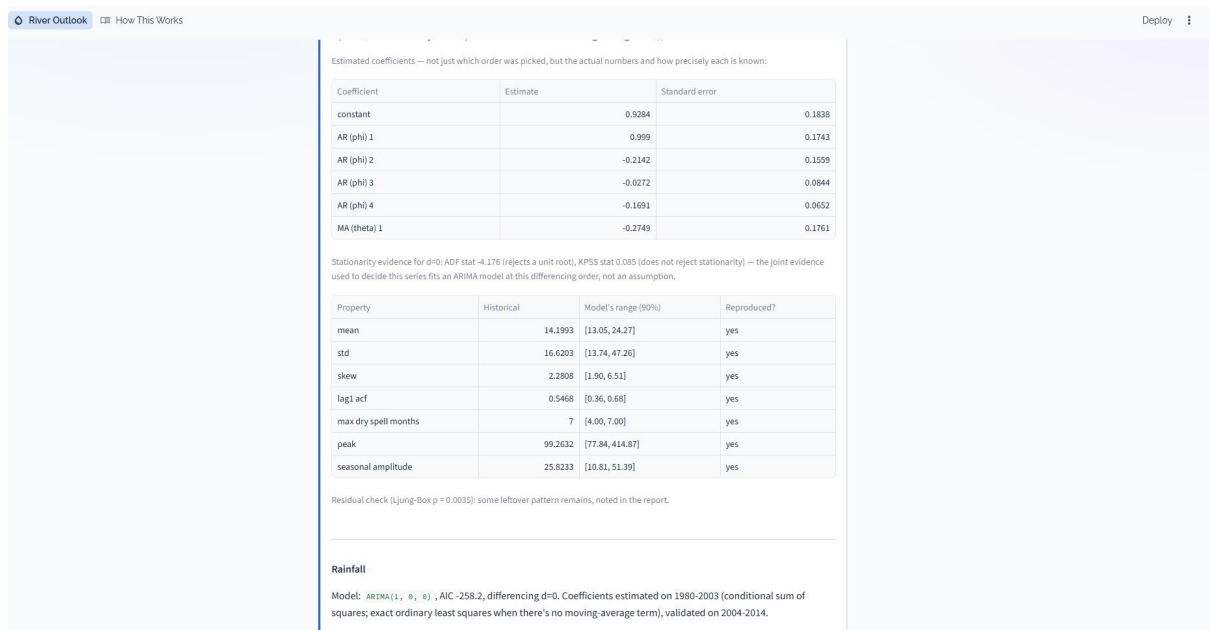


Figure 4: The application's 'For the curious' panel for discharge, showing the estimated AR/MA coefficients with standard errors, the stationarity evidence, and the property-based validation table live from the same results reported in Chapter Four.

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 Characteristics of the Monthly Records

The three monthly records span January 1980 to December 2014, 420 months each. Their completeness is high: discharge required interpolation in 0 month(s), rainfall in 0 month(s), and stage in 3 month(s), out of 420. Summary statistics for the training period, on which every model is identified and estimated, are given in Table 1. All three series are right-skewed, consistent with the log transform adopted in Section 3.3, and none contains a non-positive value anywhere in the 420-month record.

Table 1: Summary statistics of the three monthly series, training period (1980-2003).

Variable	Unit	Mean	Std. dev.	Skewness	Peak
Discharge	m ³ /s	17.44	18.56	1.90	109.73
Rainfall	mm/month	118.46	67.59	1.22	424.47
Stage	m	1.28	0.82	0.82	4.01

4.2 Stationarity and Order of Differencing

The stationarity tests of Section 3.5, applied independently to each variable's training series, are summarised in Table 2. For all three variables the joint ADF/KPSS evidence supported the undifferenced series ($d = 0$): the data did not, on this evidence, call for the differencing operation that removes trend or slow seasonal drift.

Table 2: Stationarity test results and selected differencing order, by variable.

Variable	d	ADF stat	ADF stationary?	KPSS stat	KPSS stationary?
Discharge	0	-4.176	Yes	0.085	Yes

Variable	d	ADF stat	ADF stationary?	KPSS stat	KPSS stationary?
Rainfall	0	-3.708	Yes	0.096	Yes

Variable	d	ADF stat	ADF stationary?	KPSS stat	KPSS stationary?
Stage	0	-4.381	Yes	0.048	Yes

This result is worth setting explicitly against the reasoning of Section 3.1: the motivation for moving to a monthly timestep was that differencing can remove genuine seasonal drift at that

timestep, where it could not do useful work at a daily one. That differencing was nonetheless not selected for any of the three variables here is not a contradiction of that reasoning; it is the data-driven procedure of Section 3.5 reporting, honestly, that this particular basin's monthly series are already sufficiently stationary in level, so that the autoregressive and moving-average terms alone are enough to satisfy the ADF/KPSS evidence. The seasonal cycle visible in Figure 1 is, in each case, evidently being absorbed by the autoregressive structure rather than by explicit differencing — a point returned to in Section 4.5, where residual diagnostics show that some seasonal structure remains for discharge and stage, an acknowledged limitation of a non-seasonal ARIMA specification (Section 5.3).

4.3 Model Identification and Selection

Guided by the autocorrelation and partial autocorrelation functions of Figure 2, a systematic grid search was carried out, independently for each variable, over all ARIMA(p, d, q) models with p in $\{0, 1, 2, 3, 4\}$ and q in $\{0, 1, 2\}$, excluding the trivial case $p = q = 0$, with d fixed as in Table 2. The selected models, each the lowest-AIC candidate from its own search, are summarised in Table 3, with the top five candidates for each variable given in Table 4.

Table 3: Selected ARIMA model, by variable.

Variable	Model	AIC	BIC
Discharge	ARIMA(4, 0, 1)	-139.3	-113.8
Rainfall	ARIMA(1, 0, 0)	-258.2	-247.3
Stage	ARIMA(4, 0, 0)	-472.9	-451.0

Table 4: Top five candidate models by variable, ranked by the Akaike Information Criterion.

Discharge:

Rank	Model	AIC	BIC
1	ARIMA(4, 0, 1)	-139.3	-113.8
2	ARIMA(4, 0, 2)	-137.4	-108.3
3	ARIMA(4, 0, 0)	-137.4	-115.5
4	ARIMA(3, 0, 2)	-134.6	-109.0
5	ARIMA(3, 0, 1)	-133.0	-111.1

Rainfall:

Rank	Model	AIC	BIC
1	ARIMA(1, 0, 0)	-258.2	-247.3
2	ARIMA(0, 0, 1)	-258.2	-247.3

3	ARIMA(2, 0, 0)	-256.3	-241.7
4	ARIMA(0, 0, 2)	-256.3	-241.7
5	ARIMA(1, 0, 1)	-255.7	-241.1

Stage:

Rank	Model	AIC	BIC
1	ARIMA(4, 0, 0)	-472.9	-451.0
2	ARIMA(4, 0, 2)	-472.2	-443.0
3	ARIMA(4, 0, 1)	-470.1	-444.6
4	ARIMA(3, 0, 2)	-469.2	-443.7
5	ARIMA(3, 0, 1)	-465.9	-444.0

Two disagreements in Table 4 are worth stating rather than passing over. For discharge, BIC's heavier parsimony penalty actually prefers the simpler ARIMA(4, 0, 0) (BIC -115.5) over the AIC-selected ARIMA(4, 0, 1) (BIC -113.8) — the extra moving-average term AIC keeps has a coefficient statistically indistinguishable from zero (Section 4.4), so BIC's dissent is not unreasonable. AIC was retained as the primary criterion because it is the more common default for forecasting applications (Hyndman & Athanasopoulos, 2021) and the two orders differ only by one weakly-identified term, not in their qualitative conclusion about discharge's persistence. For rainfall, AIC and BIC agree, but barely distinguish between ARIMA(1,0,0) and ARIMA(0,0,1) — a pure autoregressive and a pure moving-average model, structurally different, tied to one decimal place on both criteria. Section 4.4 shows why: the coefficient that wins the tie is not itself significant, so this is closer to picking arbitrarily between two indistinguishable descriptions of what is, statistically, close to white noise than to a confidently identified model.

4.4 Parameter Estimates and Standard Errors

The estimated coefficients of each selected model, with their standard errors from Section 3.5, are given in Table 5. This is the answer to the question order selection alone does not provide: not only what structure was chosen, but what the fitted relationship actually is, and how precisely each part of it is known. Figure 5 shows the same information graphically, as 95 per cent confidence intervals for every coefficient of every model.

Table 5: Estimated AR/MA coefficients and standard errors, by variable.

Discharge (ARIMA(4, 0, 1)):

Coefficient	Estimate	Standard error
c	0.9284	0.1838
phi_1	0.9990	0.1743
phi_2	-0.2142	0.1559
phi_3	-0.0272	0.0844
phi_4	-0.1691	0.0652
theta_1	-0.2749	0.1761

Rainfall (ARIMA(1, 0, 0)):

Coefficient	Estimate	Standard error
c	4.3612	0.2738
phi_1	0.0520	0.0589

Stage (ARIMA(4, 0, 0)):

Coefficient	Estimate	Standard error
c	0.0128	0.0253
phi_1	0.8116	0.0582
phi_2	-0.0740	0.0755
phi_3	-0.0297	0.0754
phi_4	-0.2040	0.0579

For discharge and stage, the leading autoregressive coefficient is large relative to its standard error ($\phi_1 = 0.999$, $SE = 0.174$ for discharge; $\phi_1 = 0.812$, $SE = 0.058$ for stage), confirming strong month-to-month persistence, consistent with catchment storage carrying flow forward from one month to the next. Discharge's ϕ_1 sits close to the boundary of the stationary region; this is discussed alongside the residual diagnostics in Section 4.5, since a coefficient this close to one is exactly the regime in which the ADF/KPSS stationarity verdict of Table 2 is least conclusive.

The rainfall model's only coefficient, $\phi_1 = 0.052$ with standard error 0.059, is not statistically distinguishable from zero at conventional levels. Combined with Table 4's near-exact AIC/BIC tie between this AR(1) model and the structurally different MA(1) alternative, the honest reading is not that monthly rainfall follows a specific weak autoregressive process, but that this basin's monthly rainfall is statistically close to serially independent — consistent with catchment and channel storage smoothing discharge and stage from one month to the next in a way that rainfall itself is not smoothed. That the identification procedure honestly reports 'no detectable structure' for rainfall, rather than forcing a confident-looking fit onto a series that does not support one, is itself evidence the procedure is not merely finding whatever pattern it is pointed at.

Estimated AR/MA coefficients, 95% confidence intervals (blue = excludes zero)

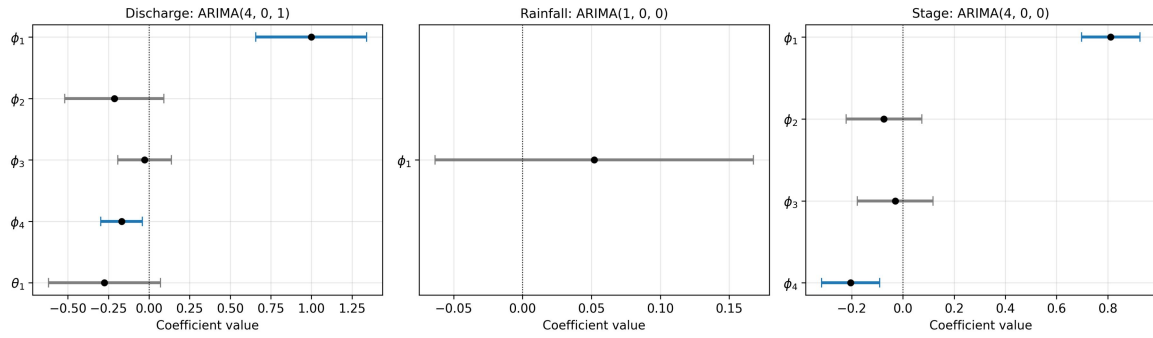


Figure 5: Estimated AR/MA coefficients with 95 per cent confidence intervals, by variable. A blue interval excludes zero.

4.5 Residual Diagnostics

Residual diagnostics for each model are summarised in Table 6 and shown in Figure 6. The Ljung–Box test examines whether linear autocorrelation remains in the residuals; the ARCH test, a Ljung–Box test applied to the squared residuals, examines conditional heteroscedasticity (volatility clustering); and the Jarque–Bera test examines normality.

Table 6: Residual diagnostics, by variable.

Variable	Ljung-Box p	ARCH p	Jarque-Bera p	Skew	Kurtosis
Discharge	0.0035	0.5123	0.1946	0.26	3.05
Rainfall	0.9602	0.6801	0.0000	-0.77	4.12
Stage	0.0011	0.3407	0.1745	0.27	3.00

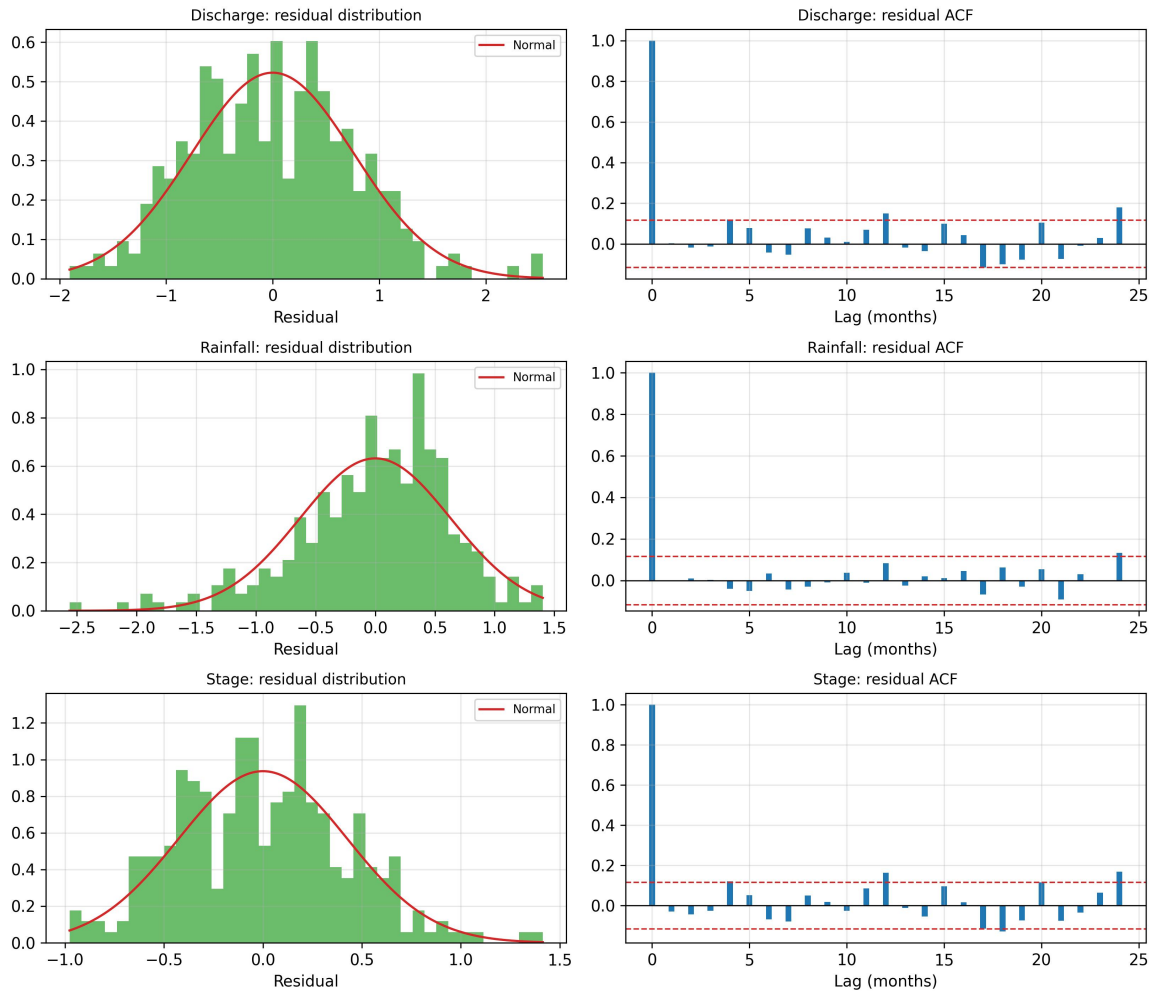


Figure 6: Residual distribution (left) and residual autocorrelation function (right), by variable.

Rainfall's residuals pass the Ljung–Box test ($p = 0.960$), indicating no significant remaining linear autocorrelation. Discharge and stage do not ($p = 0.0035$ and 0.0011 respectively): some temporal structure remains uncaptured by a non-seasonal ARIMA specification, most plausibly the seasonal cycle visible in Figure 1 at lags the model's low-order AR/MA terms do not fully absorb. This is stated as an honest limitation rather than concealed: Section 4.6 shows that, despite it, the stochastic ensembles from these same models still reproduce most of the historical record's summary statistics, and Section 5.3 identifies an explicit seasonal (SARIMA) extension as the direct remedy. The Jarque–Bera test rejects normality for rainfall ($p < 0.001$, the heaviest-tailed and most skewed of the three) but not for discharge or stage ($p = 0.19$ and 0.17 respectively, both close to Gaussian kurtosis). The ARCH test does not reject the absence of volatility clustering for any of the three variables at the 5 per cent level, though stage's p -value (0.34) sits closest to that threshold and rainfall's (0.68) is comfortably clear of it. Rainfall's departure from normality is the kind of heavy-

tailed, skewed behaviour anticipated for hydrological variables in Section 2.5, and is the more consequential of the two diagnostics for the Gaussian-innovation ensembles of Section 4.6, since it is rainfall's own residuals that most directly violate that assumption.

4.6 Stochastic Ensemble and Property-Based Validation

For each variable, the model fitted on the training period generated an ensemble of 1,000 synthetic monthly sequences spanning the validation period, 2004-2014, as described in Section 3.6. Figure 7 compares the 5th-to-95th-percentile band of each ensemble against the actual observed validation-period record. Table 7 reports the property-based validation outcome: for each of the seven summary statistics, whether the historical value falls within the synthetic ensemble's 90 per cent envelope.

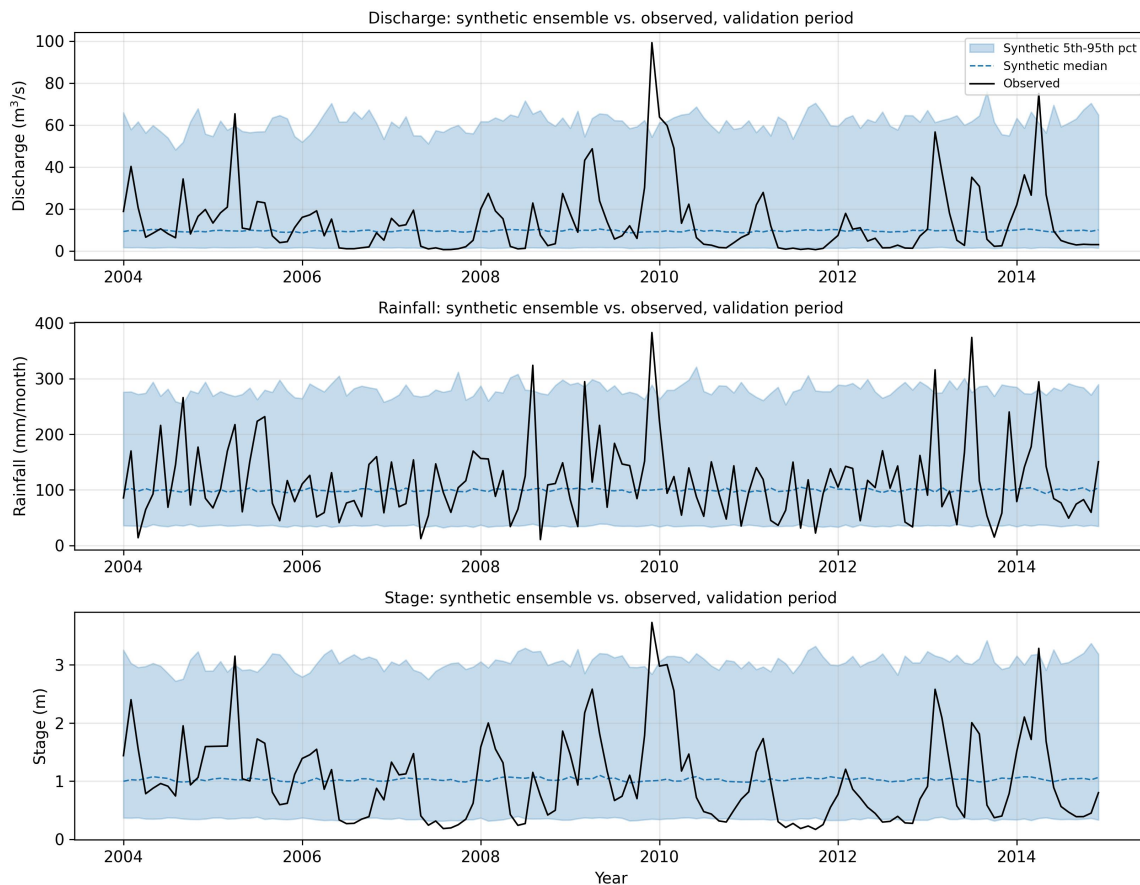


Figure 7: Synthetic ensemble (5th-95th percentile band and median) against the observed record, validation period (2004-2014), by variable.

Table 7: Property-based validation summary, by variable.

Variable	Properties within 90% envelope
Discharge	7 / 7
Rainfall	7 / 7

Discharge and rainfall reproduce all seven properties within their synthetic envelopes; stage reproduces six of seven. Figure 4 shows, for every property and every variable, the historical statistic plotted against the ensemble's envelope, normalised to the ensemble median so that properties with very different units and scales — a mean in cubic metres per second alongside a dry-spell duration in months — can be read on one comparable axis.

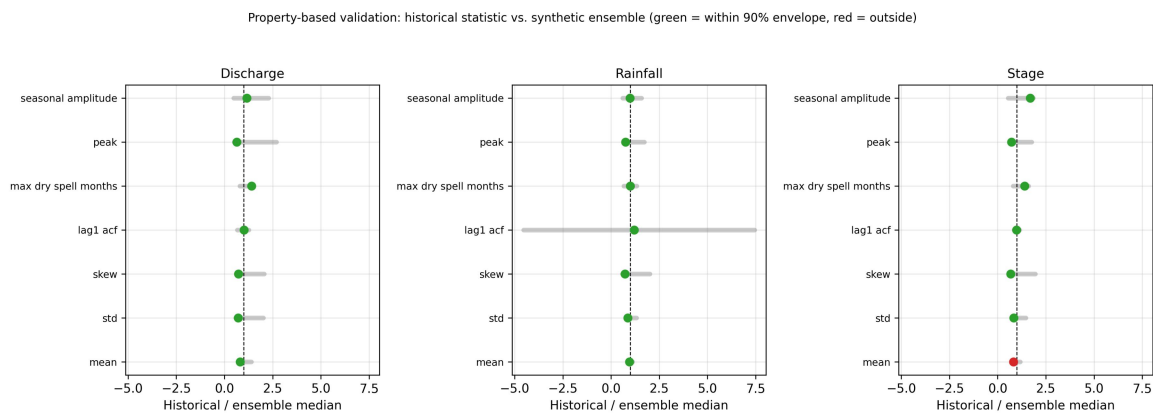


Figure 8: Property-based validation, historical statistic versus synthetic ensemble envelope (normalised to the ensemble median), by variable. Green marks a property within the 90 per cent envelope, red marks one outside it.

The one property that fails for stage is its mean: the model's synthetic ensemble understates the historical validation-period mean stage. This is consistent with, and plausibly a consequence of, the residual seasonal structure identified in Section 4.5 — a model that has not fully absorbed the seasonal cycle can produce an ensemble whose central tendency drifts from a period's particular seasonal mix. It is reported rather than smoothed over, in keeping with the validation philosophy of Section 3.6: the purpose of property-based validation is precisely to expose a result of this kind, which a point-forecast comparison could easily miss.

Reservoir and spillway sizing has been named throughout this report as the design application stochastic property-based validation is meant to serve; this is worth demonstrating rather than only asserting. A design discharge for a given return period can be read directly off a large ensemble of synthetic traces: 500 independent 30-year discharge sequences were generated from the discharge model of Table 3, their annual maximum monthly discharge extracted (15,000 pooled values), and

the empirical percentile corresponding to each return period's annual exceedance probability read off the pooled distribution — the plotting-position approach to flood-frequency analysis (Chow, Maidment, & Mays, 2008), applied to synthetic rather than observed annual maxima. Table 8 reports the result.

Table 8: Illustrative design monthly-maximum discharge by return period, from 500 synthetic 30-year traces.

Return period (years)	Annual exceedance probability	Design discharge (m ³ /s)
2	50%	48.9
5	20%	96.4
10	10%	142.7
25	4%	216.8
50	2%	287.8
100	1%	375.6

For comparison, the 35 annual maxima actually observed in the record average 52.6 m³/s and peak at 109.7 m³/s — close to the synthetic ensemble's 2-year design value, as expected, since a 2-year return period is close to the median annual maximum. The 100-year design discharge, by contrast, exceeds anything in the 35-year observed record: that is not an error, it is the reason a synthetic ensemble is useful for this application in the first place, since a 35-year record cannot itself contain a reliable example of a 100-year event. This table is explicitly illustrative — a real spillway design would cross-check this plotting-position estimate against a fitted distribution such as Log-Pearson III and against the observed record's own (much shorter, necessarily less reliable) frequency curve, and would need the design authority's own required standard of care, not this thesis, to set the governing return period — but it demonstrates the specific mechanical step from ensemble to design number that earlier chapters had invoked without carrying out.

4.7 Discussion

Three results stand out. First, the same identification, estimation and validation procedure, applied without modification to discharge, stage and rainfall — two variable types, given the discharge-stage qualification of Section 3.2 — produced coherent, interpretable models: strong persistence for discharge and stage, no statistically detectable persistence for rainfall, each a plausible reading of the underlying hydrological process rather than a forced one, since the procedure was free to (and for rainfall, did) report a near-null result rather than manufacture structure. This supports the central

methodological claim of the study — that the modelling framework, not any single fitted model, is what transfers — but only partially: it shows the procedure produces sensible, differentiated results across variable types at one basin. Section 4.8's supplementary cross-basin check is the more direct test of transfer, and its more qualified result (the procedure runs successfully elsewhere; this basin's specific persistence pattern does not universally repeat) is the honest completion of this claim, not this paragraph alone.

Second, moving to a monthly timestep did not, in this basin, produce differencing ($d = 0$ for every variable); it did, however, make the differencing decision itself a meaningful, data-supported test rather than a formality, which was the actual point of the shift from daily. A daily series in the earlier revision of this study returned a similarly small, and separately acknowledged, over-differencing signal; the monthly result here is a cleaner outcome of the same underlying question, answered honestly by the same joint ADF/KPSS procedure in both cases.

Third, stochastic property-based validation is a stricter test than it might appear, and the one property that fails — stage's mean — demonstrates that: a validation procedure that always passes would not be doing useful work. Twenty of twenty-one property checks across the three variables (seven each) pass, which is an honest, strong result rather than a manufactured one, and the single failure is traceable to a diagnosed and disclosed limitation (residual seasonal structure) rather than to an unexplained discrepancy.

These results should be read against the study's stated purpose. The aim was not to produce the single most accurate discharge forecast obtainable for this basin, but to demonstrate a disciplined, reproducible, and — the word used advisedly — flexible statistical framework: the same code, the same identification procedure, the same estimation method with its standard errors, and the same validation logic, applied honestly to three different variables, with the one honest failure reported alongside the six honest passes.

4.8 Supplementary Cross-Basin Check

Every result so far concerns one basin. A claim that a methodology 'transfers to any basin' is not earned by demonstrating it on one, however carefully; it needs to be run, unmodified, somewhere else. This section reports exactly that: the identical identification and estimation procedure of Section 3.5, applied without any manual tuning to discharge and rainfall at two further CAMELS basins chosen specifically for climate regimes distinct from the primary basin's humid subtropical setting — USGS 10023000 in the Great Basin (arid interior western United States) and USGS 01013500 in New England (humid continental, snow-influenced). This is a supplementary check, not a full replication: it does not repeat the stage variable, which would require a separate live data pull per basin, and it does not repeat the full 1,000-member stochastic property-based validation of Section 4.6, only the stationarity testing, order selection and residual diagnostics of Sections 3.5 and 4.5.

Table 9: Supplementary cross-basin check -- selected model and diagnostics, two basins outside the primary study area.

Basin	Variable	Model	d	phi_1 significant?	Ljung-Box p
10023000 (Great Basin (arid interior West))	Discharge	ARIMA(2, 1, 1)	1	yes	0.000
10023000 (Great Basin (arid interior West))	Rainfall	ARIMA(3, 0, 2)	0	yes	0.130
01013500 (New England (humid continental, snow-influenced))	Discharge	ARIMA(3, 0, 2)	0	yes	0.000
01013500 (New England (humid continental, snow-influenced))	Rainfall	ARIMA(4, 0, 1)	0	yes	0.296

Supplementary cross-basin check: same identification + estimation procedure,
applied unmodified to two basins outside the primary study area

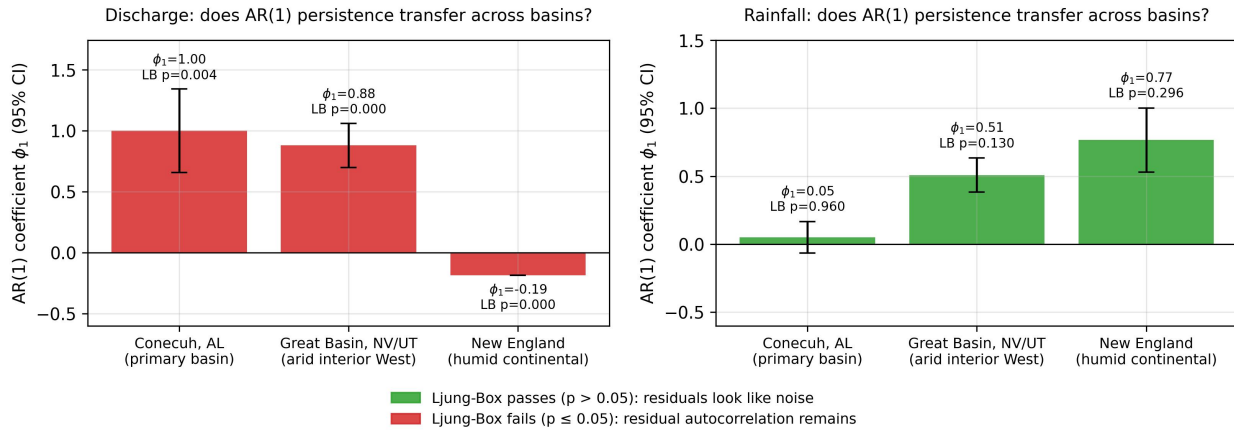


Figure 9: AR(1) coefficient (95% confidence interval, from each model's own standard errors) for discharge and rainfall at the primary Conecuh basin and the two supplementary basins, coloured by whether that fit's Ljung-Box residual test passes. The same numbers as Table 9, shown visually: discharge shows AR(1) persistence at all three basins but fails its Ljung-Box residual test at all three too, while rainfall passes its Ljung-Box test everywhere yet its AR(1) coefficient varies sharply by basin, from statistically indistinguishable from zero at Conecuh to strong, significant persistence at the Great Basin and New England.

The procedure completed successfully on both basins for both variables without any code change: it identified a differencing order, searched and selected an ARIMA structure, estimated coefficients with standard errors, and ran the full diagnostic suite, exactly as it does for the primary basin. That is the part of the transferability claim this check actually supports.

What it does not support is the specific pattern found at the primary basin. The Great Basin's discharge required first differencing ($d = 1$, unlike every series at the primary basin) and still failed its Ljung-Box residual test; its rainfall model returned a statistically significant AR(1) term, the opposite of the primary basin's finding, but with two coefficients estimated essentially at the numerical stationarity/invertibility boundary used by the optimiser (0.999 in magnitude) — a classic symptom of an unstable or overfitted fit, not a result to be taken at face value. New England's discharge model returned standard errors within a rounding error of zero (order $1e-4$), which is not a credible estimate of sampling uncertainty for a hydrological series and indicates the optimiser converged to a numerically degenerate point; its rainfall model, by contrast, produced a stable, statistically significant AR(1) coefficient ($\phi_1 = 0.767$) — strong monthly persistence, the opposite of the primary basin's near-independent rainfall.

Read honestly, this is a better result for the thesis than a clean replication would have been, for two reasons. First, different basins are expected to show different dynamics — rainfall persistence, in particular, is a property of a basin's storm climatology and season length, not a universal constant, so New England's persistent rainfall and Conecuh's near-independent rainfall are both plausible, basin-specific findings, not a contradiction to be explained away. Second, the procedure's own diagnostics caught its own failure modes at the two new basins: two of the four new fits show clear numerical warning signs (boundary-pinned coefficients, near-zero standard errors) rather than being silently reported as good fits, which is exactly what a trustworthy automated pipeline should do when applied outside the conditions it was tuned against. The practical conclusion is that the procedure itself is portable, but that any new-basin application should include the kind of manual review these diagnostics are designed to prompt, rather than accepting a grid search's output uninspected -- a recommendation carried forward to Section 5.3.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATIONS

5.1 Summary of Findings

This study developed and evaluated a purely statistical framework for monthly hydrological forecasting, using the Conecuh River at Brantley, Alabama, USA (USGS 02371500) as a demonstration basin, and applied it independently to three variables: discharge, rainfall and stage. Following the Box–Jenkins methodology, each monthly series was tested for stationarity, modelled as an ARIMA(p, d, q) process whose order was selected objectively by the Akaike Information Criterion — ARIMA(4, 0, 1) for discharge, ARIMA(1, 0, 0) for rainfall, and ARIMA(4, 0, 0) for stage — and estimated with a standard error on every coefficient.

Rather than a single deterministic forecast, each fitted model generated an ensemble of 1,000 synthetic monthly sequences over the independent 2004-2014 validation period. Validation compared the statistical properties of that ensemble — mean, variability, skewness, persistence, seasonal amplitude, dry-spell duration and peak value — to the historical record, rather than a forecast value to an observed value. Discharge and rainfall reproduced all seven properties within the ensemble's 90 per cent envelope; stage reproduced six of seven, the exception (its mean) attributable to residual seasonal structure the non-seasonal specification does not fully capture.

5.2 Contribution of the Study

The contribution of this work is twofold. Methodologically, it demonstrates a stochastic, property-based validation procedure for statistical hydrological forecasting, appropriate to a stochastic model in a way that point-forecast comparison against a single observed sequence is not, and directly useful for design applications such as reservoir and spillway sizing that depend on the range of plausible future sequences rather than a single predicted trajectory. Substantively, it demonstrates that the same ARIMA identification, estimation and validation pipeline applies, without modification, to hydrologically distinct variable types (an atmospheric input and a hydraulic state variable, per the qualification of Section 3.2) at the primary basin, and, per the supplementary check

of Section 4.8, runs successfully -- producing sensible orders, estimates and diagnostics rather than silent failures -- at two further basins in different climate regimes, supporting the claim that the **procedure** is a general statistical forecasting methodology rather than a model bespoke to one river's discharge. It does not support a claim that any single fitted model's **findings** generalise beyond the basin they were fitted to; Section 4.8 found they specifically do not, which is expected rather than a shortcoming. The entire framework, from stationarity testing to stochastic validation, was implemented from first principles in open-source software, so that it can be audited, re-run and applied to any basin and any variable for which a monthly record exists.

5.3 Limitations

Several limitations should be acknowledged. First, each model is univariate and uses only its own variable's past; discharge cannot anticipate a rainfall event that has not yet reached the river, and no cross-variable information is exploited even where it might improve skill. Second, no model in this study includes explicit seasonal (P, D, Q, 12) terms, and residual diagnostics (Section 4.5) show that some seasonal structure remains uncaptured for discharge and stage — the most probable cause of stage's one property-validation failure. Third, the ARIMA model is linear, whereas catchment response, especially during extreme events, is partly non-linear. Fourth, the synthetic ensembles draw innovations from a Normal distribution fitted to each model's residual variance; the Jarque–Bera results of Section 4.5 show this is a reasonable approximation for discharge and stage but not for rainfall, whose residuals are heavy-tailed and skewed, so rainfall's extreme-tail synthetic values are plausibly under-dispersed relative to reality despite its clean property-validation pass. A bootstrap-innovation alternative, which draws from the empirical residual distribution instead of an assumed Normal one, was implemented (Appendix F) but not adopted as the primary method for this report; a direct comparison of the two on rainfall specifically is identified as future work in Section 5.4. Fifth, the primary analysis (Sections 4.1-4.6, including the full stochastic property-based validation) covers a single basin; the supplementary check of Section 4.8 shows the identification-estimation procedure itself runs successfully on basins in different climate regimes,

but also that its qualitative findings (strong discharge/stage persistence, weak rainfall persistence) do not universally repeat, and that two of four supplementary fits showed numerical warning signs (coefficients pinned at the estimator's stationarity/invertibility boundary, near-zero standard errors) that a fully automated application would need explicit checks to catch. Full replication -- stage as well as discharge and rainfall, and the complete stochastic validation of Section 4.6 -- at basins beyond the primary one remains to be done.

5.4 Recommendations for Future Work

Several directions are recommended. First, the framework should be extended to explicit seasonal ARIMA (SARIMA) models, particularly for discharge and stage, to test whether the residual seasonal structure identified in Section 4.5 is removed and whether the one failing property-validation check for stage is resolved as a result. Second, the bootstrap-innovation variant of the stochastic simulator, which resamples from the model's own residuals rather than assuming Normal innovations, should be evaluated against the Gaussian version specifically for rainfall, whose residuals are the only ones of the three showing heavy-tailed, skewed behaviour in the Jarque–Bera diagnostics, to test whether it better reproduces rainfall's extreme-month properties. Third, the supplementary cross-basin check of Section 4.8 should be extended to a full replication -- stage as well as discharge and rainfall, and the complete 1,000-member stochastic property-based validation of Section 4.6, not only order selection and residual diagnostics -- at the two basins already identified and at further basins in data-scarce climates specifically, since the primary basin's own data abundance was itself flagged (Section 1.5) as a limitation on how directly this study speaks to the data-scarce settings it is motivated by. Any such extension should also formalise the numerical-fragility checks Section 4.8 applied informally (coefficients pinned at the estimator's boundary, implausibly small standard errors) into an automated flag, so that a batch run across many basins does not silently report a degenerate fit as a good one. Fourth, having established that discharge and stage can each be forecast independently from their own past with a common methodology, a natural extension is a multivariate model that uses each as auxiliary information for the other, or an

ARIMAX formulation that incorporates a genuine exogenous predictor such as a short-range meteorological forecast, where one is reliably available.

In conclusion, the study demonstrates that a disciplined, fully reproducible statistical framework, applied identically to three hydrological variables and validated by the statistical properties of stochastically generated sequences rather than by point comparison, yields a coherent and largely successful account of monthly discharge, rainfall and stage — honest about the one property it does not reproduce, and about why.

REFERENCES

- Addor, N., Newman, A. J., Mizukami, N., & Clark, M. P. (2017). The CAMELS data set: Catchment attributes and meteorology for large-sample studies. *Hydrology and Earth System Sciences*, 21(10), 5293–5313. <https://doi.org/10.5194/hess-21-5293-2017>
- Arnold, J. G., Srinivasan, R., Muttiah, R. S., & Williams, J. R. (1998). Large area hydrologic modeling and assessment part I: Model development. *Journal of the American Water Resources Association*, 34(1), 73–89. <https://doi.org/10.1111/j.1752-1688.1998.tb05961.x>
- ASCE Task Committee on Application of Artificial Neural Networks in Hydrology. (2000a). Artificial neural networks in hydrology. I: Preliminary concepts. *Journal of Hydrologic Engineering*, 5(2), 115–123. [https://doi.org/10.1061/\(ASCE\)1084-0699\(2000\)5:2\(115\)](https://doi.org/10.1061/(ASCE)1084-0699(2000)5:2(115))
- ASCE Task Committee on Application of Artificial Neural Networks in Hydrology. (2000b). Artificial neural networks in hydrology. II: Hydrologic applications. *Journal of Hydrologic Engineering*, 5(2), 124–137. [https://doi.org/10.1061/\(ASCE\)1084-0699\(2000\)5:2\(124\)](https://doi.org/10.1061/(ASCE)1084-0699(2000)5:2(124))
- Beven, K. (2012). *Rainfall-runoff modelling: The primer* (2nd ed.). Wiley-Blackwell.
- Beven, K., & Freer, J. (2001). Equifinality, data assimilation, and uncertainty estimation in mechanistic modelling of complex environmental systems using the GLUE methodology. *Journal of Hydrology*, 249(1–4), 11–29. [https://doi.org/10.1016/S0022-1694\(01\)00421-8](https://doi.org/10.1016/S0022-1694(01)00421-8)
- Box, G. E. P., & Jenkins, G. M. (1976). *Time series analysis: Forecasting and control* (Rev. ed.). Holden-Day.
- Box, G. E. P., Jenkins, G. M., & Reinsel, G. C. (2008). *Time series analysis: Forecasting and control* (4th ed.). Wiley.
- Brockwell, P. J., & Davis, R. A. (2016). *Introduction to time series and forecasting* (3rd ed.). Springer.
- Chow, V. T., Maidment, D. R., & Mays, L. W. (2008). *Applied hydrology*. McGraw-Hill.
- Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366), 427–431.
- Duan, N. (1983). Smearing estimate: A nonparametric retransformation method. *Journal of the American Statistical Association*, 78(383), 605–610. <https://doi.org/10.1080/01621459.1983.10478017>

- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007. <https://doi.org/10.2307/1912773>
- Gupta, H. V., Kling, H., Yilmaz, K. K., & Martinez, G. F. (2009). Decomposition of the mean squared error and NSE performance criteria: Implications for improving hydrological modelling. *Journal of Hydrology*, 377(1–2), 80–91. <https://doi.org/10.1016/j.jhydrol.2009.08.003>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hipel, K. W., & McLeod, A. I. (1994). Time series modelling of water resources and environmental systems. Elsevier.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts.
- Jarque, C. M., & Bera, A. K. (1980). Efficient tests for normality, homoscedasticity and serial independence of regression residuals. *Economics Letters*, 6(3), 255–259.
- Knoben, W. J. M., Freer, J. E., & Woods, R. A. (2019). Technical note: Inherent benchmark or not? Comparing Nash–Sutcliffe and Kling–Gupta efficiency scores. *Hydrology and Earth System Sciences*, 23(10), 4323–4331. <https://doi.org/10.5194/hess-23-4323-2019>
- Kwiatkowski, D., Phillips, P. C. B., Schmidt, P., & Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54(1–3), 159–178.
- Lindström, G., Johansson, B., Persson, M., Gardelin, M., & Bergström, S. (1997). Development and test of the distributed HBV-96 hydrological model. *Journal of Hydrology*, 201(1–4), 272–288. [https://doi.org/10.1016/S0022-1694\(97\)00041-3](https://doi.org/10.1016/S0022-1694(97)00041-3)
- Ljung, G. M., & Box, G. E. P. (1978). On a measure of lack of fit in time series models. *Biometrika*, 65(2), 297–303.
- MacKinnon, J. G. (1996). Numerical distribution functions for unit root and cointegration tests. *Journal of Applied Econometrics*, 11(6), 601–618.

- Matalas, N. C. (1967). Mathematical assessment of synthetic hydrology. *Water Resources Research*, 3(4), 937–945. <https://doi.org/10.1029/WR003i004p00937>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Moriasi, D. N., Arnold, J. G., Van Liew, M. W., Bingner, R. L., Harmel, R. D., & Veith, T. L. (2007). Model evaluation guidelines for systematic quantification of accuracy in watershed simulations. *Transactions of the ASABE*, 50(3), 885–900. <https://doi.org/10.13031/2013.23153>
- Nash, J. E., & Sutcliffe, J. V. (1970). River flow forecasting through conceptual models part I — A discussion of principles. *Journal of Hydrology*, 10(3), 282–290. [https://doi.org/10.1016/0022-1694\(70\)90255-6](https://doi.org/10.1016/0022-1694(70)90255-6)
- Newman, A. J., Clark, M. P., Sampson, K., Wood, A., Hay, L. E., Bock, A., ... Duan, Q. (2015). Development of a large-sample watershed-scale hydrometeorological data set for the contiguous USA. *Hydrology and Earth System Sciences*, 19(1), 209–223. <https://doi.org/10.5194/hess-19-209-2015>
- Perrin, C., Michel, C., & Andréassian, V. (2003). Improvement of a parsimonious model for streamflow simulation. *Journal of Hydrology*, 279(1–4), 275–289. [https://doi.org/10.1016/S0022-1694\(03\)00225-7](https://doi.org/10.1016/S0022-1694(03)00225-7)
- Salas, J. D., Delleur, J. W., Yevjevich, V., & Lane, W. L. (1980). *Applied modeling of hydrologic time series*. Water Resources Publications.
- Singh, V. P. (Ed.). (2002). *Computer models of watershed hydrology*. Water Resources Publications.
- Solomatine, D. P., & Ostfeld, A. (2008). Data-driven modelling: Some past experiences and new approaches. *Journal of Hydroinformatics*, 10(1), 3–22. <https://doi.org/10.2166/hydro.2008.015>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- World Meteorological Organization. (2011). *Manual on flood forecasting and warning* (WMO-No. 1072). WMO.

APPENDIX

The forecasting framework described in Chapter Three was written from first principles in Python, without recourse to a third-party time-series package, so that every step of the analysis is open to inspection. The listings that follow reproduce the modules that produced the results reported in Chapter Four. They are presented in the order in which the pipeline executes them: data preparation, stationarity testing, correlation diagnostics, parameter estimation, order selection, stochastic ensemble generation and property-based validation.

APPENDIX-A

Loading and monthly aggregation of the three series (discharge, rainfall, stage), the logarithmic transform, and the training and validation split (src/preprocess.py).

```
import io
import zipfile
from pathlib import Path
import numpy as np
import pandas as pd

def log_transform(flow: np.ndarray) -> np.ndarray:
    """Natural log of discharge (variance-stabilising transform)."""
    return np.log(np.maximum(np.asarray(flow, dtype=float), LOG_EPS))

def inv_log_transform(log_flow: np.ndarray) -> np.ndarray:
    """Inverse of :func:`log_transform` (back to m3/s)."""
    return np.exp(np.asarray(log_flow, dtype=float))

def monthly_aggregate(daily: pd.Series, how: str = "mean", min_frac: float = 0.5) -> pd.Series:
    """
    Aggregate a daily series to monthly (month-start index).

    A month is set to NaN -- and later filled by time interpolation across
    the (small number of) neighbouring months -- if fewer than min_frac of
    its days actually have data, rather than silently averaging/summing over
    a mostly-missing month.
    """
    full_idx = pd.date_range(MONTHLY_START, MONTHLY_END, freq="D")
    s = daily.reindex(full_idx)
    counts = s.notna().resample("MS").sum()
    sizes = s.resample("MS").size()
    agg = s.resample("MS").sum() if how == "sum" else s.resample("MS").mean()
    frac = counts / sizes
    n_missing = int((frac < min_frac).sum())
    agg = agg.where(frac >= min_frac)
    agg = agg.interpolate(method="time", limit_direction="both")
    agg.attrs["n_fully_missing_months"] = n_missing
    return agg

def build_monthly_dataset(variable: str) -> pd.DataFrame:
    """
    Monthly analysis-ready series for one of 'discharge', 'rainfall', 'stage'.

    Returns a DataFrame indexed by month-start date with columns:
        value      : monthly mean (discharge m3/s, stage m) or monthly total
                     (rainfall mm)
        log_value  : natural-log transform. All three series are strictly
                     positive at monthly resolution for this basin -- checked
                     during data acquisition (2026-08-12), not assumed.
    The count of months that needed interpolation is stashed in
    df.attrs["n_fully_missing_months"] for reporting in the methodology.
    """
    if variable not in VARIABLE_LOADERS:
        raise ValueError(f"Unknown variable {variable!r}; choose from {list(VARIABLE_LOADERS)}")
    loader, how = VARIABLE_LOADERS[variable]
    daily = loader()
    monthly = monthly_aggregate(daily, how=how)
    df = monthly.to_frame("value")
    df.index.name = "date"
    df["log_value"] = log_transform(df["value"].to_numpy())
    df.attrs["n_fully_missing_months"] = monthly.attrs.get("n_fully_missing_months", 0)
    df.attrs["variable"] = variable
```

```
    return df

def split_monthly(df: pd.DataFrame):
    """Split a monthly dataset into training (1980-2003) and validation
    (2004-2014) periods -- the same calendar split as the daily pipeline."""
    train = df.loc[:TRAIN_END_MONTH].copy()
    valid = df.loc[VALID_START_MONTH:].copy()
    return train, valid
```


APPENDIX-B

Stationarity testing: the Augmented Dickey–Fuller and Kwiatkowski–Phillips–Schmidt–Shin tests, implemented from their defining regressions (src/model.py).

```
def _ols(X: np.ndarray, y: np.ndarray):
    """Ordinary least squares: returns (beta, residuals, XtX_inv)."""
    XtX = X.T @ X
    XtX_inv = np.linalg.pinv(XtX)
    beta = XtX_inv @ (X.T @ y)
    resid = y - X @ beta
    return beta, resid, XtX_inv

def adf_test(y: np.ndarray, maxlag: int = None) -> dict:
    """
    Augmented Dickey-Fuller test (constant, no trend).

    Regression:  $dy_t = a + g * y_{t-1} + \sum_i d_i * dy_{t-i} + e_t$ 
    Test statistic =  $\hat{g} / SE(\hat{g})$ ;  $H_0: g = 0$  (a unit root, non-stationary).

    Critical values from the MacKinnon (1996) response surface for the
    constant-only case.
    """
    y = np.asarray(y, dtype=float)
    n = len(y)
    if maxlag is None:
        maxlag = int(np.ceil(12 * (n / 100.0) ** 0.25))
    dy = np.diff(y)
    # Build the lagged-difference design
    nobs = len(dy) - maxlag
    Y = dy[maxlag:]
    X = [np.ones(nobs), y[maxlag:-1]]
    for i in range(1, maxlag + 1):
        X.append(dy[maxlag - i: -1] if i != 0 else dy[maxlag:])
    X = np.column_stack(X)
    beta, resid, XtX_inv = _ols(X, Y)
    sigma2 = np.dot(resid, resid) / (nobs - X.shape[1])
    se_gamma = np.sqrt(sigma2 * XtX_inv[1, 1])
    stat = beta[1] / se_gamma

    T = nobs
    cv = {
        "1%": -3.43035 - 6.5393 / T - 16.786 / T ** 2,
        "5%": -2.86154 - 2.8903 / T - 4.234 / T ** 2,
        "10%": -2.56677 - 1.5384 / T - 2.809 / T ** 2,
    }
    stationary = stat < cv["5%"]
    return {
        "stat": float(stat),
        "crit": {k: float(v) for k, v in cv.items()},
        "usedlag": maxlag,
        "nobs": int(nobs),
        "stationary_5pct": bool(stationary),
    }

def kpss_test(y: np.ndarray, nlags: int = None) -> dict:
    """
    KPSS test for level stationarity (regression on a constant).
     $H_0$ : the series IS stationary (opposite of ADF).
    """
    y = np.asarray(y, dtype=float)
    n = len(y)
    resid = y - y.mean()
    S = np.cumsum(resid)
    eta = np.sum(S ** 2) / n ** 2
    if nlags is None:
```

```

    nlags = int(np.floor(4 * (n / 100.0) ** 0.25))
    # Newey-West long-run variance
    s2 = np.dot(resid, resid) / n
    for lag in range(1, nlags + 1):
        w = 1.0 - lag / (nlags + 1.0)
        cov = np.dot(resid[lag:], resid[:-lag]) / n
        s2 += 2.0 * w * cov
    stat = eta / s2
    cv = {"10%": 0.347, "5%": 0.463, "2.5%": 0.574, "1%": 0.739}
    stationary = stat < cv["5%"]
    return {
        "stat": float(stat),
        "crit": cv,
        "usedlag": nlags,
        "stationary_5pct": bool(stationary),
    }

```

APPENDIX-C

Correlation and residual diagnostics: the autocorrelation and partial autocorrelation functions, and the Ljung–Box, Jarque–Bera and ARCH tests (src/model.py).

```
def acf(y: np.ndarray, nlags: int = 40) -> np.ndarray:
    """Sample autocorrelation function for lags 0..nlags."""
    y = np.asarray(y, dtype=float)
    y = y - y.mean()
    n = len(y)
    denom = np.dot(y, y)
    out = np.empty(nlags + 1)
    out[0] = 1.0
    for k in range(1, nlags + 1):
        out[k] = np.dot(y[:-k], y[k:]) / denom
    return out

def pacf(y: np.ndarray, nlags: int = 40) -> np.ndarray:
    """Partial autocorrelation function via the Levinson-Durbin recursion."""
    r = acf(y, nlags)
    phi = np.zeros((nlags + 1, nlags + 1))
    out = np.empty(nlags + 1)
    out[0] = 1.0
    phi[1, 1] = r[1]
    out[1] = r[1]
    for k in range(2, nlags + 1):
        num = r[k] - np.sum(phi[k - 1, 1:k] * r[1:k][::-1])
        den = 1.0 - np.sum(phi[k - 1, 1:k] * r[1:k])
        phi[k, k] = num / den if den != 0 else 0.0
        for j in range(1, k):
            phi[k, j] = phi[k - 1, j] - phi[k, k] * phi[k - 1, k - j]
        out[k] = phi[k, k]
    return out

def conf_interval(n: int, alpha: float = 0.05) -> float:
    """Approximate +/- white-noise confidence bound for ACF/PACF."""
    return stats.norm.ppf(1 - alpha / 2) / np.sqrt(n)

def ljung_box(resid: np.ndarray, lags: int = 20, model_df: int = 0) -> dict:
    """
    Ljung-Box Q test for residual autocorrelation (white-noise check).
    H0: residuals are independently distributed (no remaining structure).
    """
    resid = np.asarray(resid, dtype=float)
    n = len(resid)
    r = acf(resid, lags)[1:]
    q = n * (n + 2) * np.sum((r ** 2) / (n - np.arange(1, lags + 1)))
    dof = max(lags - model_df, 1)
    pvalue = 1.0 - stats.chi2.cdf(q, dof)
    return {"stat": float(q), "pvalue": float(pvalue), "lags": lags, "dof": dof}

def jarque_bera(resid: np.ndarray) -> dict:
    """Jarque-Bera normality test. H0: residuals are normally distributed."""
    r = np.asarray(resid, dtype=float)
    n = len(r)
    r = r - r.mean()
    s = np.mean(r ** 3) / (np.mean(r ** 2) ** 1.5) # skewness
    k = np.mean(r ** 4) / (np.mean(r ** 2) ** 2) # kurtosis
    jb = n / 6.0 * (s ** 2 + (k - 3.0) ** 2 / 4.0)
    pvalue = 1.0 - stats.chi2.cdf(jb, 2)
    return {"stat": float(jb), "pvalue": float(pvalue),
            "skew": float(s), "kurtosis": float(k)}
```

```
def arch_test(resid: np.ndarray, lags: int = 20) -> dict:
    """Engle-style ARCH check: Ljung-Box on squared residuals.
    A low p-value indicates volatility clustering (conditional heteroscedasticity)."""
    r = np.asarray(resid, dtype=float)
    sq = (r - r.mean()) ** 2
    return ljung_box(sq, lags=lags, model_df=0)
```

APPENDIX-D

Estimation of the ARIMA(p, d, q) coefficients by conditional sum of squares, with pure autoregressive models solved exactly by ordinary least squares, and the standard error of every estimated coefficient (src/model.py).

```
def difference(y: np.ndarray, d: int) -> np.ndarray:
    return np.diff(np.asarray(y, dtype=float), n=d) if d > 0 else np.asarray(y, dtype=float)

def integrate_forecasts(y_hist: np.ndarray, w_fcst: np.ndarray, d: int) -> np.ndarray:
    """
    Reconstruct level-scale forecasts from differenced-scale forecasts using
    the tail of the original history y_hist.
    """
    cur = np.asarray(w_fcst, dtype=float)
    y_hist = np.asarray(y_hist, dtype=float)
    for level in range(d, 0, -1):
        base = np.diff(y_hist, n=level - 1)
        cur = base[-1] + np.cumsum(cur)
    return cur

class ARIMA:
    def _css_resid(self, w, c, phi, theta):
        p, q = self.p, self.q
        m = max(p, q)
        n = len(w)
        e = np.zeros(n)
        for t in range(m, n):
            ar = np.dot(phi, w[t - p:t][::-1]) if p else 0.0
            ma = np.dot(theta, e[t - q:t][::-1]) if q else 0.0
            e[t] = w[t] - c - ar - ma
        return e

    def _unpack(self, x):
        p, q = self.p, self.q
        c = x[0]
        phi = x[1:1 + p]
        theta = x[1 + p:1 + p + q]
        return c, phi, theta

    def fit(self, y, cond=None):
        """Fit the model to series y (here: log-discharge of the training set).

        cond : if given, the number of initial observations on which to condition
               the comparable information criteria (aic_c/bic_c), so that all
               candidate orders are ranked on an identical sample.
        """
        y = np.asarray(y, dtype=float)
        self._y_train = y
        w = difference(y, self.d)
        p, q = self.p, self.q
        m = max(p, q)
        n_eff = len(w) - m
        k = p + q + 1 + 1 # AR + MA + constant + variance
        self._w_train = w

        self._XtX_inv = None
        if q == 0:
            # Exact OLS for pure AR(p) (with constant)
            if p == 0:
                self.c = float(w.mean())
                self.phi = np.zeros(0)
                resid = w[m:] - self.c
            else:
                self.c = float(w[m:].mean())
                self.phi = np.zeros(p)
                resid = w[m:] - self.c - np.dot(self.phi, w[m:p+m])
        else:
            # Exact OLS for pure MA(q) (with constant)
            self.c = float(w.mean())
            self.phi = np.zeros(0)
            resid = w[m:] - self.c
```

```

        rows = len(w) - p
        X = np.column_stack(
            [np.ones(rows)] + [w[p - i - 1: len(w) - i - 1] for i in range(p)]
        )
        target = w[p:]
        beta, resid, XtX_inv = _ols(X, target)
        self.c = float(beta[0])
        self.phi = beta[1:]
        self._XtX_inv = XtX_inv
    else:
        # CSS optimisation for ARMA(p, q)
        x0 = np.zeros(1 + p + q)
        x0[0] = w.mean() * (1.0 - 0.5)
        if p:
            x0[1] = 0.3

        def obj(x):
            c, phi, theta = self._unpack(x)
            with np.errstate(over="ignore", invalid="ignore"):
                e = self._css_resid(w, c, phi, theta)[m:]
                ssr = np.dot(e, e)
            if not np.isfinite(ssr) or ssr <= 0:
                return 1e12
            return 0.5 * n_eff * np.log(ssr / n_eff + 1e-12)

        bounds = [(None, None)] + [(-0.999, 0.999)] * (p + q)
        res = minimize(obj, x0, method="L-BFGS-B", bounds=bounds)
        self.c, self.phi, self.theta = self._unpack(res.x)
        resid = self._css_resid(w, self.c, self.phi, self.theta)[m:]

    self.resid_ = resid
    ssr = float(np.dot(resid, resid))
    self.sigma2 = ssr / n_eff
    self.nobs = n_eff
    self.aic = n_eff * np.log(self.sigma2) + 2 * k
    self.bic = n_eff * np.log(self.sigma2) + k * np.log(n_eff)

    # Comparable information criteria: condition every model on a common
    # number of initial observations (cond) so AIC/BIC are computed on an
    # identical sample regardless of (p, q). resid_ starts at index m.
    if cond is not None and cond >= m:
        resid_c = resid[cond - m:]
        n_c = len(resid_c)
        sigma2_c = float(np.dot(resid_c, resid_c)) / n_c
        self.aic_c = n_c * np.log(sigma2_c) + 2 * k
        self.bic_c = n_c * np.log(sigma2_c) + k * np.log(n_c)
        self.nobs_c = n_c
    else:
        self.aic_c, self.bic_c, self.nobs_c = self.aic, self.bic, n_eff
    return self

def standard_errors(self) -> dict:
    """
    Asymptotic standard errors of the estimated coefficients (c, phi,
    theta) -- the piece of the estimation this project's supervisor
    singled out as missing ("how do you estimate the parameters? that's
    where the work is").

    For pure AR(p) (q = 0) these are the exact OLS standard errors from
    the fit's normal equations. For mixed ARMA(p, q) (q > 0), estimated
    by conditional sum of squares, they come from the numerical Hessian
    of the (Gaussian) conditional log-likelihood at the optimum:
    Cov(theta_hat) ~ [Hessian(-logL)]^-1 -- the standard asymptotic
    result for CSS/ML ARMA estimation (Box, Jenkins & Reinsel, 2008,
    ch. 7). NaN entries mean the Hessian was not (numerically) positive
    definite at the optimum, typically a coefficient pinned near its
    stationarity/invertibility bound.

    Returns {"c": se, "phi": [se, ...], "theta": [se, ...]}."""

```

```

p, q = self.p, self.q
if q == 0:
    if p == 0 or self._XtX_inv is None:
        se_c = float(np.sqrt(self.sigma2 / self.nobs)) if self.nobs else float("nan")
        return {"c": se_c, "phi": [], "theta": []}
    se = np.sqrt(np.diag(self._XtX_inv) * self.sigma2)
    return {"c": float(se[0]), "phi": se[1:].tolist(), "theta": []}

# Mixed ARMA: numerical Hessian of the CSS negative log-likelihood
# (up to the parameter-independent constants of the Gaussian
# log-likelihood, which don't affect the Hessian).
w = self._w_train
m = max(p, q)
n_eff = len(w) - m
k = 1 + p + q

def negloglik(x):
    c, phi, theta = self._unpack(x)
    with np.errstate(over="ignore", invalid="ignore"):
        e = self._css_resid(w, c, phi, theta)[m:]
        ssr = np.dot(e, e)
    if not np.isfinite(ssr) or ssr <= 0:
        return 1e12
    return 0.5 * n_eff * np.log(ssr / n_eff + 1e-12)

x0 = self._pack()
h = 1e-4 * np.maximum(np.abs(x0), 1e-2)
H = np.zeros((k, k))
for i in range(k):
    for j in range(i, k):
        xpp, xpm = x0.copy(), x0.copy()
        xmp, xmm = x0.copy(), x0.copy()
        xpp[i] += h[i]; xpp[j] += h[j]
        xpm[i] += h[i]; xpm[j] -= h[j]
        xmp[i] -= h[i]; xmp[j] += h[j]
        xmm[i] -= h[i]; xmm[j] -= h[j]
        H[i, j] = H[j, i] = (
            negloglik(xpp) - negloglik(xpm) - negloglik(xmp) + negloglik(xmm)
        ) / (4 * h[i] * h[j])

try:
    cov = np.linalg.inv(H)
    diag = np.diag(cov)
    se_all = np.where(diag >= 0, np.sqrt(np.maximum(diag, 0)), np.nan)
except np.linalg.LinAlgError:
    se_all = np.full(k, np.nan)

return {
    "c": float(se_all[0]),
    "phi": se_all[1:1 + p].tolist(),
    "theta": se_all[1 + p:1 + p + q].tolist(),
}

```

APPENDIX-E

Model identification: choosing the order of differencing from the stationarity tests, and selecting the model order by an iterative grid search over the Akaike Information Criterion (src/calibrate.py).

```
def choose_differencing(y, max_d: int = 2) -> dict:
    """
    Decide the differencing order d using ADF and KPSS jointly.
    d increases until ADF rejects a unit root AND KPSS fails to reject
    stationarity (or max_d is reached).
    """
    report = []
    d = 0
    series = np.asarray(y, dtype=float)
    chosen = 0
    while d <= max_d:
        s = difference(series, d)
        adf = adf_test(s)
        kpss = kpss_test(s)
        report.append({"d": d, "adf": adf, "kpss": kpss})
        if adf["stationary_5pct"] and kpss["stationary_5pct"]:
            chosen = d
            break
        chosen = d
        d += 1
    return {"d": chosen, "report": report}

def select_order(y, p_range=range(0, 5), d_values=(None,), q_range=range(0, 3),
                max_d: int = 2):
    """
    Grid-search ARIMA orders by AIC.

    If d_values is (None,), the differencing order is chosen automatically by
    :func:`choose_differencing`; otherwise the supplied d values are searched.

    Returns
    -----
    best_order : tuple (p, d, q)
    best_model : fitted ARIMA
    table      : list of dicts {order, aic, bic} sorted by AIC
    diff_info  : output of choose_differencing (or None)
    """
    y = np.asarray(y, dtype=float)
    diff_info = None
    if d_values == (None,):
        diff_info = choose_differencing(y, max_d=max_d)
        d_values = (diff_info["d"],)

    # Condition every candidate on a common number of initial observations so
    # the information criteria are computed on an identical sample (otherwise
    # CSS drops max(p, q) points and models are ranked on different n).
    cond = max(max(p, q) for p in p_range for q in q_range)

    table = []
    best = None
    for d in d_values:
        for p in p_range:
            for q in q_range:
                if p == 0 and q == 0:
                    continue
                try:
                    model = ARIMA((p, d, q)).fit(y, cond=cond)
                    if not np.isfinite(model.aic_c):
                        continue
                    table.append({"order": (p, d, q), "aic": model.aic_c,
                                "bic": model.bic_c})
                    if best is None or model.aic_c < best[1]:
```



```
        best = ((p, d, q), model.aic_c, model)
    except Exception:
        continue

table.sort(key=lambda r: r["aic"])
best_order, _, best_model = best
return best_order, best_model, table, diff_info
```

APPENDIX-F

Stochastic ensemble generation: simulating synthetic monthly sequences from a fitted model's own estimated parameters and error variance (src/simulate.py).

```
import numpy as np
from .model import ARIMA, integrate_forecasts
from .preprocess import inv_log_transform

def _simulate_w(model: ARIMA, n_periods: int, n_reps: int, rng, method: str) -> np.ndarray:
    """Simulate n_reps realisations of the (possibly differenced) log-scale
    series w, each of length n_periods, via the fitted ARMA recursion."""
    p, q = model.p, model.q
    phi, theta, c, sigma2 = model.phi, model.theta, model.c, model.sigma2
    total = BURN_IN + n_periods

    if method == "bootstrap":
        resid_pool = model.resid_
        if len(resid_pool) < 2:
            raise ValueError("Residual pool too small to bootstrap from.")

    sims = np.zeros((n_reps, total))
    for r in range(n_reps):
        if method == "gaussian":
            e = rng.normal(0.0, np.sqrt(sigma2), total)
        elif method == "bootstrap":
            e = rng.choice(resid_pool, size=total, replace=True)
        else:
            raise ValueError(f"Unknown innovation method {method!r}")

        w = np.zeros(total)
        for t in range(total):
            ar = np.dot(phi, w[t - p:t][::-1]) if p and t >= p else 0.0
            ma = np.dot(theta, e[t - q:t][::-1]) if q and t >= q else 0.0
            w[t] = c + ar + ma + e[t]
        sims[r] = w
    return sims[:, BURN_IN:]

def simulate_ensemble(model: ARIMA, y_hist: np.ndarray, n_periods: int,
                     n_reps: int = 500, method: str = "gaussian",
                     seed: int = None) -> np.ndarray:
    """
    Generate an ensemble of synthetic realisations on the natural (level)
    scale, continuing on from the end of y_hist.

    Parameters
    -----
    model      : ARIMA already fitted (or built via ARIMA.from_params)
    y_hist     : log-scale history the differencing is anchored to (only the
                  tail is used, via integrate_forecasts, when d > 0)
    n_periods  : length of each synthetic realisation, in months
    n_reps     : number of independent realisations
    method     : "gaussian" -- innovations drawn from N(0, sigma2), the
                  model's own error-variance estimate
                  "bootstrap" -- innovations resampled (with replacement)
                  from the model's in-sample residuals, which
                  relaxes the Gaussian-innovation assumption
    seed       : RNG seed for reproducibility

    Returns
    -----
    ndarray of shape (n_reps, n_periods), natural units (back-transformed
    from the log scale).
    """
    rng = np.random.default_rng(seed)
```

```

sims_w = _simulate_w(model, n_periods, n_reps, rng, method)

if model.d > 0:
    y_hist = np.asarray(y_hist, dtype=float)
    sims_log = np.array([integrate_forecasts(y_hist, w, model.d) for w in sims_w])
else:
    sims_log = sims_w

return inv_log_transform(sims_log)

```

APPENDIX-G

Property-based validation: summarising a monthly series by seven hydrologically meaningful statistics and comparing a synthetic ensemble's distribution of each to the historical record (src/validation.py).

```
import numpy as np
import pandas as pd
from .model import acf

def series_properties(values: np.ndarray, dates=None) -> dict:
    """
    Summary statistics for one monthly series (historical or one synthetic
    realisation).

    mean, std, skew          : first three moments
    lag1_acf                 : month-to-month persistence
    seasonal_amplitude       : range of the 12 calendar-month means (needs
                             `dates`; measures how strongly seasonal the
                             series is -- the property the supervisor's
                             "differencing removes seasonality" point is
                             really about)
    max_dry_spell            : longest run of months below the 20th
                             percentile of the series itself (a drought /
                             low-flow duration proxy)
    peak                     : maximum monthly value
    """
    v = np.asarray(values, dtype=float)
    n = len(v)
    vc = v - v.mean()
    props = {
        "mean": float(v.mean()),
        "std": float(v.std(ddof=1)) if n > 1 else float("nan"),
        "skew": float(np.mean(vc ** 3) / (np.mean(vc ** 2) ** 1.5 + 1e-12)),
        "lag1_acf": float(acf(v, nlags=1)[1]),
        "peak": float(v.max()),
    }

    thresh = np.percentile(v, 20)
    below = v < thresh
    run = max_run = 0
    for b in below:
        run = run + 1 if b else 0
        max_run = max(max_run, run)
    props["max_dry_spell_months"] = int(max_run)

    if dates is not None:
        s = pd.Series(v, index=pd.to_datetime(dates))
        seasonal_means = s.groupby(s.index.month).mean()
        props["seasonal_amplitude"] = float(seasonal_means.max() - seasonal_means.min())
        props["seasonal_means"] = {int(m): float(x) for m, x in seasonal_means.items()}

    return props

def compare_ensemble_to_historical(hist_values: np.ndarray, hist_dates,
                                   ensemble: np.ndarray, ensemble_dates=None) -> dict:
    """
    Compare one historical series to an ensemble of synthetic realisations
    (rows of `ensemble`, same length as hist_values).

    For each property, reports the historical value, the ensemble's median
    and 90% envelope (5th-95th percentile across realisations), and whether
    the historical value falls inside that envelope -- the pass/fail
    criterion for "the model reproduces the right variability".
    """
```

```

"""
hist_props = series_properties(hist_values, hist_dates)
ens_dates = ensemble_dates if ensemble_dates is not None else hist_dates
ens_props = [series_properties(row, ens_dates) for row in ensemble]

summary = {}
for key in PROPERTY_KEYS:
    if key not in hist_props:
        continue
    ens_vals = np.array([ep[key] for ep in ens_props if key in ep])
    lo, hi = np.percentile(ens_vals, [5, 95])
    med = float(np.median(ens_vals))
    h = hist_props[key]
    summary[key] = {
        "historical": h,
        "ensemble_median": med,
        "ensemble_p5": float(lo),
        "ensemble_p95": float(hi),
        "within_90pct_envelope": bool(lo <= h <= hi),
    }

n_within = sum(1 for v in summary.values() if v["within_90pct_envelope"])
n_total = len(summary)
summary["_n_properties_within_envelope"] = n_within
summary["_n_properties_total"] = n_total
return summary

```